คู่มือฉบับสมบูรณ์สำหรับการสร้างและปรับปรุงระบบ Retrieval-Augmented Generation (RAG) สำหรับนักวิจัยมือใหม่

บทนำ: ทำความเข้าใจ RAG สำหรับนักวิจัยมือใหม่

ในยุคปัจจุบันที่โมเดลภาษาขนาดใหญ่ (Large Language Models - LLMs) มีบทบาทสำคัญในการประมวลผลภาษาธรรมชาติ (Natural Language Processing - NLP) เทคนิค Retrieval-Augmented Generation (RAG) ได้กลายเป็นแนวทางที่ปฏิวัติวงการ โดยเฉพาะอย่างยิ่งสำหรับงานที่ต้องการความแม่นยำ ความทันสมัย และความสามารถในการตรวจสอบข้อมูล RAG ผสานรวมความสามารถของ LLMs เข้ากับระบบดึงข้อมูลภายนอก ทำให้การสร้างข้อความมีความถูกต้องและสอดคล้องกับบริบทมากขึ้นในหลากหลายแอปพลิเคชัน [1]

RAG คืออะไรและทำไมถึงสำคัญ

Retrieval-Augmented Generation (RAG) เป็นกรอบการทำงานของ AI ที่เชื่อมโยง LLMs เข้ากับแหล่งความรู้ภายนอก เช่น เอกสาร ฐานข้อมูล หรือข้อมูลที่มีโครงสร้าง เพื่อปรับปรุงประสิทธิภาพของโมเดลและสร้างผลลัพธ์ที่แม่นยำและเกี่ยวข้องกับบริบทมากขึ้น.[2] โดยทั่วไปแล้ว LLMs จะสร้างการตอบสนองโดยอาศัยข้อมูลที่ได้รับการฝึกฝนมาเท่านั้น ซึ่งอาจนำไปสู่ข้อมูลที่ล้าสมัย ไม่ถูกต้อง หรือที่เรียกว่า "การหลอนข้อมูล" (hallucinations).[3] RAG เข้ามาแก้ไขข้อจำกัดนี้โดยการดึงข้อมูลที่เกี่ยวข้องแบบไดนามิก ณ เวลาที่สร้างข้อความ ทำให้โมเดลสามารถผลิตผลลัพธ์ที่มีข้อมูลครบถ้วนและถูกต้องตามบริบทมากขึ้น.[2]
ความสำคัญของ RAG อยู่ที่ความสามารถในการแก้ปัญหาพื้นฐานของ LLMs ที่ต้องพึ่งพาชุดข้อมูลการฝึกที่คงที่ ซึ่งทำให้ความรู้ของโมเดลถูกจำกัดอยู่เพียง "จุดตัดความรู้" (knowledge cutoff) ของข้อมูลที่ใช้ในการฝึก.[4] เมื่อมีการสอบถามเกี่ยวกับเหตุการณ์ล่าสุดหรือความรู้เฉพาะทางที่ไม่มีอยู่ในข้อมูลการฝึก โมเดลอาจสร้างข้อมูลที่ไม่ถูกต้องหรือล้าสมัยขึ้นมา.[3] RAG จัดการกับปัญหานี้โดยตรงด้วยการนำขั้นตอนการดึงข้อมูลแบบไดนามิกเข้ามาใช้ การที่ข้อมูลการฝึกของ LLM มีลักษณะคงที่นั้นเป็นสาเหตุ

ของการเกิดการหลอนข้อมูล และการดึงข้อมูลแบบไดนามิกของ RAG ก็ช่วยแก้ไขปัญหานี้โดยการให้บริบทภายนอกที่ทันสมัยและสามารถตรวจสอบได้ การดำเนินการนี้เปลี่ยน LLMs จากคลังความรู้แบบคงที่ไปสู่ "กลไกการให้เหตุผล" แบบไดนามิกที่สามารถโต้ตอบและสังเคราะห์ข้อมูลจากโลกที่เปลี่ยนแปลงตลอดเวลา ทำให้ LLMs มีประโยชน์และใช้งานได้จริงมากขึ้นสำหรับแอปพลิเคชันระดับองค์กรในโลกแห่งความเป็นจริง.[4]

หลักการพื้นฐานและวัตถุประสงค์ของ RAG

แก่นแท้ของ RAG คือแนวทางที่เน้นความรู้ ซึ่งให้ความสำคัญกับความรู้ภายนอกเป็นปัจจัยสำคัญในการปรับปรุงการสร้างภาษา.[2] ด้วยการรวมข้อมูลที่เกี่ยวข้อง ข้อมูลแบบเรียลไทม์ และข้อมูลที่มีโครงสร้าง โมเดล RAG สามารถเพิ่มความสามารถในการสร้างเนื้อหาที่ถูกต้องตามบริบทและมีข้อเท็จจริงที่อ้างอิงได้อย่างมาก.[2]

วัตถุประสงค์หลักของ RAG ประกอบด้วย:

- **ความเข้าใจเจตนาของผู้ใช้ที่แม่นยำ:** การทำความเข้าใจเจตนาของผู้ใช้ได้อย่างถูกต้องมีความสำคัญอย่างยิ่งสำหรับโมเดล RAG ในการสร้างการตอบสนองที่เกี่ยวข้องกับความหมายและเหมาะสมกับบริบท.[2] การดำเนินการนี้ต้องอาศัยการวิเคราะห์คำถามของผู้ใช้ที่อาจกำกวมหรือมีความหมายโดยนัย ซึ่งมักจะขาดหายไป การทำความเข้าใจเจตนาของผู้ใช้ที่แม่นยำนี้เป็นสิ่งสำคัญอย่างยิ่งสำหรับโมเดล RAG ในการสร้างการตอบสนองที่ทั้งเกี่ยวข้องกับความหมายและเหมาะสมกับบริบท ซึ่งจำเป็นต้องมีการวิเคราะห์คำถามของผู้ใช้ที่อาจกำกวมหรือมีความหมายโดยนัย การดำเนินการนี้เหนือกว่าการจับคู่คำหลักแบบดั้งเดิม โดยเจาะลึกการวิเคราะห์ความหมายที่ลึกซึ้งยิ่งขึ้นเพื่อจับเจตนาที่แท้จริงของผู้ใช้.[2]
- **การอ้างอิงข้อเท็จจริง:** วัตถุประสงค์หลักคือการอ้างอิงผลลัพธ์ของ LLM กับข้อมูลภายนอกที่เป็นข้อเท็จจริงและเชื่อถือได้ ซึ่งช่วยลดการหลอนข้อมูลและเพิ่มความน่าเชื่อถือได้อย่างมาก.[3]
- **ความทันสมัย:** การให้การเข้าถึงข้อมูลที่เป็นปัจจุบันและแบบเรียลไทม์ เพื่อเอาชนะข้อจำกัดด้านความรู้ของ LLMs ที่ได้รับการ

ฝึกฝนล่วงหน้า.[4]

- **ความสามารถในการตรวจสอบ:** การทำให้ผู้ใช้สามารถเชื่อถือและตรวจสอบการตอบสนองได้ โดยการระบุแหล่งที่มาอย่างละเอียด.[5]

การเน้นย้ำถึง "ความเข้าใจเจตนาของผู้ใช้ที่แม่นยำ" [2] ในฐานะวัตถุประสงค์พื้นฐานเผยให้เห็นว่าความสำเร็จของ RAG ไม่ได้ขึ้นอยู่กับส่วนประกอบการดึงข้อมูลและการสร้างข้อความเพียงอย่างเดียว แต่ยังขึ้นอยู่กับความเข้าใจเริ่มต้นในความต้องการของผู้ใช้ด้วย หากระบบตีความคำถามผิดพลาด แม้การดึงข้อมูลที่ไม่เกี่ยวข้องอย่างสมบูรณ์ก็ยังนำไปสู่คำตอบที่ไม่ดีได้ สิ่งนี้บ่งชี้ถึงความสัมพันธ์เชิงสาเหตุ: ความเข้าใจเจตนาที่ไม่ดีนำไปสู่การกำหนดคำถามที่ไม่ดี ซึ่งนำไปสู่การดึงข้อมูลที่ไม่เกี่ยวข้อง และท้ายที่สุดก็คือการสร้างข้อความที่ไม่ถูกต้องหรือไม่เป็นประโยชน์ ดังนั้น การประมวลผลคำถามในระยะเริ่มต้น (เช่น การเขียนคำถามใหม่ การจัดกลุ่มเจตนา) จึงเป็นปัจจัยสำคัญที่มักถูกประเมินต่ำไปต่อประสิทธิภาพโดยรวมของ RAG.[2]

ประโยชน์หลักของ RAG

RAG มอบประโยชน์ที่สำคัญหลายประการในการนำ AI ไปใช้งานจริง:

- **ลดการหลอนข้อมูลและเพิ่มความถูกต้องของข้อเท็จจริง:** ด้วยการอ้างอิงโมเดลกับบริบทที่ดึงมา RAG ช่วยลดการสร้างข้อมูลที่ไม่ถูกต้องหรือไร้สาระได้อย่างมาก.[3]
- **การเข้าถึงข้อมูลที่ทันสมัยและเฉพาะโดเมน:** RAG ช่วยให้ LLMs สามารถเข้าถึงฐานความรู้ภายนอก รวมถึงข้อมูลภายในองค์กร วารสารวิชาการ ฟีดข้อมูลแบบเรียลไทม์ และชุดข้อมูลเฉพาะทาง ซึ่งเอาชนะข้อจำกัดด้านความรู้ของ LLMs ที่ได้รับการฝึกฝนล่วงหน้า.[4]
- **การนำ AI ไปใช้งานอย่างคุ้มค่าและปรับขนาดได้:** RAG เป็นทางเลือกที่คุ้มค่ากว่าการฝึกฝนใหม่หรือการปรับแต่ง LLM ซึ่งมีค่าใช้จ่ายสูงและใช้ทรัพยากรมากสำหรับการรวมข้อมูลใหม่.[4]
- **เพิ่มความน่าเชื่อถือของผู้ใช้:** ระบบ RAG สามารถระบุแหล่งที่มาได้ ทำให้ผู้ใช้สามารถตรวจสอบความถูกต้องของการตอบสนองที่สร้างขึ้น.[5]

- **ขยายกรณีการใช้งาน:** RAG ช่วยให้ LLMs สามารถจัดการกับคำถามที่ซับซ้อนได้หลากหลายขึ้น และรวมข้อมูลจากหลายแหล่ง ทำให้มีแอปพลิเคชันที่กว้างขวางขึ้น.[2]
- **เพิ่มการควบคุมของนักพัฒนาและการบำรุงรักษาโมเดล:** นักพัฒนาสามารถปรับแหล่งข้อมูลที่โมเดลเข้าถึงได้อย่างง่ายดาย ทำให้การบำรุงรักษาและการปรับตำแหน่งสำหรับงานต่างๆ ง่ายขึ้น.[5]
- **ความปลอดภัยของข้อมูลที่มากขึ้น:** RAG รักษาการแยกส่วนระหว่างโมเดลและความรู้ภายนอก ทำให้สามารถให้และเพิกถอนการเข้าถึงข้อมูลที่เป็นกรรมสิทธิ์ได้โดยไม่ต้องรวมเข้ากับการฝึกโมเดล.[5]

สถาปัตยกรรมและขั้นตอนการทำงานของ RAG

ส่วนนี้จะเจาะลึกสถาปัตยกรรมพื้นฐานของ RAG โดยให้รายละเอียดส่วนประกอบหลักและขั้นตอนตามลำดับที่เกี่ยวข้องกับการสร้างและใช้งานระบบ RAG

ส่วนประกอบหลัก: Retriever และ Generator

โมเดล RAG ประกอบด้วยส่วนประกอบหลักสองส่วน ได้แก่ **ตัวดึงเอกสาร (document retriever)** และ **โมเดลภาษาขนาดใหญ่ (LLM)** ซึ่งทำหน้าที่เป็นตัวสร้าง (generator).[3] สิ่งที่โดดเด่นคือกระบวนการร่วมกันที่การดึงข้อมูลและการสร้างข้อความเชื่อมโยงกัน ทำให้โมเดลสามารถพิจารณาเอกสารหลายฉบับพร้อมกันได้.[7]

- **Retriever (ตัวดึงข้อมูล):**
  - **หน้าที่:** มีหน้าที่ค้นหาข้อมูลที่เกี่ยวข้องมากที่สุดจากชุดเอกสารขนาดใหญ่โดยอิงตามคำถามที่ป้อนเข้ามา.[3] ตัวดึงข้อมูลสามารถจับความสัมพันธ์ทางความหมายที่ซับซ้อนระหว่างคำถามและเอกสารได้.[7]
  - **กลไก:** แปลงเอกสารและคำถามให้เป็นตัวแทนเชิงตัวเลขที่มีมิติสูงที่เรียกว่า **embeddings** (หรือเวกเตอร์).[3] embeddings เหล่านี้มักถูกจัดเก็บไว้ใน **ฐานข้อมูลเวกเตอร์ (vector database)** หรือ **vector store** ซึ่งได้รับการปรับให้เหมาะสม

สำหรับการค้นหาความคล้ายคลึงกันอย่างมีประสิทธิภาพ.[3] จากนั้นตัวดึงข้อมูลจะค้นหาเวกเตอร์เอกสารที่คล้ายคลึงกันมากที่สุดกับเวกเตอร์คำถาม.[6]
    - **การปรับแต่ง:** จำนวนเอกสารที่ดึงมา (numCandidates, limit) สามารถปรับได้ตามความต้องการของงาน.[7]
- **Generator (ตัวสร้างข้อความ):**
    - **หน้าที่:** โดยทั่วไปคือโมเดล Transformer ขนาดใหญ่ (เช่น GPT-3.5, GPT-4, Llama2, Falcon, PaLM, BERT).[7] บทบาทของมันคือการรับคำถามที่ป้อนเข้ามาและเอกสารที่ดึงมา และสร้างการตอบสนองที่สอดคล้องและถูกต้อง.[3]
    - **กลไก:** เอกสารที่ดึงมาและคำถามต้นฉบับจะถูกนำมารวมกัน (เสริมข้อมูล) และป้อนเข้าสู่ตัวสร้างเป็นข้อมูลนำเข้าเดียว บริบทที่เสริมนี้ช่วยให้ LLM สร้างการตอบสนองที่มีข้อมูลครบถ้วนและแม่นยำยิ่งขึ้น ลดการหลอนข้อมูล.[3]

การทำงานร่วมกันระหว่าง Retriever และ Generator ไม่ใช่เพียงแค่การส่งผ่านข้อมูลตามลำดับเท่านั้น ผลลัพธ์ของตัวดึงข้อมูล (เอกสารที่เกี่ยวข้อง) เป็นสาเหตุโดยตรงที่ทำให้ตัวสร้างข้อความผลิตการตอบสนองที่มีการอ้างอิง.[3] อย่างไรก็ตาม คุณภาพของการทำงานร่วมกันนี้ขึ้นอยู่กับการจัดเรียงความหมายที่ทำได้ผ่าน embeddings อย่างมาก หาก embeddings ไม่สามารถจับความหมายหรือความสัมพันธ์ที่แท้จริงได้ เอกสารที่ "คล้ายคลึงกันที่สุด" ก็ยังอาจไม่เกี่ยวข้อง ซึ่งนำไปสู่ "การสร้างข้อความที่มีการอ้างอิงแต่ไม่ตรงประเด็นหรือไม่ถูกต้อง".[4] สิ่งนี้เน้นย้ำว่าการเลือกโมเดล embedding และคุณภาพของ vector space เป็นปัจจัยสำคัญต้นน้ำที่ส่งผลต่อความสำเร็จของไปป์ไลน์ RAG ทั้งหมด

ขั้นตอนการสร้างระบบ RAG

กระบวนการ RAG โดยทั่วไปประกอบด้วยหลายขั้นตอน ซึ่งมักจะแบ่งออกเป็น: Data Pipeline (การทำดัชนีแบบออฟไลน์) และ RAG Chain (การดึงข้อมูล การเสริมข้อมูล การสร้างข้อความแบบออนไลน์).[8]

- **การเตรียมข้อมูล: การโหลด, การแบ่งส่วน, และการจัดเก็บ**

(Data Preparation: Loading, Splitting, and Storing/Indexing)

- **ขั้นตอนที่ 1: การจัดหาและเตรียมข้อมูล (การโหลดและการทำความสะอาด):** เกี่ยวข้องกับการนำเข้าข้อมูลดิบ (เช่น ไฟล์ข้อความ, PDF, เว็บไซต์, ฐานข้อมูล, API) เข้าสู่ไปป์ไลน์ RAG.[8] Document Loaders (เช่น CheerioWebBaseLoader สำหรับเนื้อหาเว็บ) ใช้ในการโหลดข้อมูลและส่งคืนอ็อบเจกต์ Document.[11] มีการบังคับใช้ความสมบูรณ์และคุณภาพของข้อมูลโดยการลบค่าที่ขาดหายไปหรือคุณลักษณะที่ไม่เกี่ยวข้อง.[12]
- **ขั้นตอนที่ 2: การแบ่งส่วนข้อความ (Chunking):** เอกสารขนาดใหญ่จะถูกแบ่งออกเป็น "ส่วนย่อย" ที่จัดการได้ง่ายขึ้น.[3] สิ่งนี้มีความสำคัญอย่างยิ่งเนื่องจากส่วนย่อยขนาดใหญ่อาจค้นหาได้ยากและอาจเกินขีดจำกัดหน้าต่างบริบทของ LLM.[11] RecursiveCharacterTextSplitter เป็นวิธีการที่แนะนำ.[11] การเลือกขนาดส่วนย่อยและการทับซ้อนเป็นข้อตัดสินใจที่สำคัญในการออกแบบ.[11]
- **ขั้นตอนที่ 3: การสร้าง Embeddings:** แต่ละส่วนย่อยของข้อความจะถูกแปลงเป็นตัวแทนเวกเตอร์เชิงตัวเลข (embeddings) โดยใช้โมเดล embedding.[3] เวกเตอร์เหล่านี้จะจับความหมายทางความหมายและความสัมพันธ์ของข้อความ.[3] ตัวอย่าง: โมเดล embedding gte-large.[12]
- **ขั้นตอนที่ 4: การจัดเก็บ/การทำดัชนี (การตั้งค่าฐานข้อมูลเวกเตอร์):** embeddings ที่สร้างขึ้นจะถูกจัดเก็บและทำดัชนีใน **ฐานข้อมูลเวกเตอร์ (vector database)** หรือ **vector store**.[3] สิ่งนี้ช่วยให้การค้นหาความคล้ายคลึงกันของเวกเตอร์มีประสิทธิภาพในภายหลัง.[4] ตัวอย่างเช่น MongoDB Atlas [12], Pinecone, Weaviate, Elasticsearch, Chroma, FAISS, Qdrant, Milvus.[3] จำเป็นต้องสร้างดัชนีการค้นหาเวกเตอร์.[12]

- **การดึงข้อมูล (Retrieval)**
    - เมื่อผู้ใช้ส่งคำถามเข้ามา คำถามนั้นจะถูกแปลงเป็น embedding ก่อน.[3] จากนั้นตัวดึงข้อมูลจะทำการค้นหาความคล้ายคลึงกันในฐานข้อมูลเวกเตอร์เพื่อค้นหาส่วนย่อยของเอกสารที่เกี่ยวข้องมากที่สุด.[3]
- **การเสริมข้อมูล (Augmentation)**
    - ข้อมูลที่เกี่ยวข้องที่ดึงมาจะถูกนำมารวมกับคำถามต้นฉบับของผู้ใช้เพื่อสร้าง "prompt ที่ถูกเสริม".[3] ขั้นตอนนี้จะช่วยเพิ่มบริบทให้กับ LLM.[4]
- **การสร้างข้อความ (Generation)**
    - prompt ที่ถูกเสริมจะถูกป้อนเข้าสู่ LLM (ตัวสร้าง) ซึ่งจะสร้างการตอบสนองสุดท้าย.[3] LLM ใช้บริบทเพิ่มเติมเพื่อสร้างผลลัพธ์ที่มีข้อมูลครบถ้วน แม่นยำ และเกี่ยวข้องกับบริบทมากขึ้น.[4]

ลักษณะการทำงานตามลำดับของขั้นตอนเหล่านี้ (การโหลด -> การแบ่งส่วน -> การสร้าง Embedding -> การจัดเก็บ -> การดึงข้อมูล -> การเสริมข้อมูล -> การสร้างข้อความ) ก่อให้เกิดไปป์ไลน์ที่สำคัญ การตัดสินใจที่ผิดพลาดหรือไม่เหมาะสมในขั้นตอนใดๆ ก่อนหน้าจะส่งผลกระทบต่อคุณภาพของขั้นตอนถัดไป ตัวอย่างเช่น การแบ่งส่วนข้อมูลที่ไม่ดี (Splitting) อาจนำไปสู่บริบทที่แตกแยก ส่งผลให้การดึงข้อมูลไม่เกี่ยวข้อง (Retrieval) และท้ายที่สุดก็คือคำตอบที่สร้างขึ้นไม่ดี (Generation) สิ่งนี้เน้นย้ำว่าการเพิ่มประสิทธิภาพ RAG เป็นกระบวนการที่ครอบคลุม ไม่ใช่แค่การเลือก "LLM ที่ดีที่สุด" รูปแบบที่ซ่อนอยู่คือคุณภาพของขั้นตอนการทำดัชนี "ออฟไลน์" (การโหลด การแบ่งส่วน การจัดเก็บ) มีความสำคัญเท่าเทียมกับขั้นตอนการดึงข้อมูลและการสร้างข้อความ "ออนไลน์" สำหรับประสิทธิภาพโดยรวมของระบบ

ตาราง 1: ขั้นตอนการทำงานหลักของระบบ RAG และส่วนประกอบสำคัญ

Stage (ขั้นตอน)	Step (ขั้นตอน	Description	Key

	ย่อย)	(คำอธิบาย)	Components/Tools (ส่วนประกอบ/เครื่องมือสำคัญ)
Data Ingestion/Indexing (การนำเข้า/ทำดัชนีข้อมูล)	Loading (การโหลด)	การนำเข้าข้อมูลดิบจากแหล่งต่างๆ (ไฟล์, PDF, เว็บไซต์, ฐานข้อมูล).[8]	Document Loaders (เช่น CheerioWebBaseLoader) [11]
	Splitting (การแบ่งส่วน)	การแบ่งเอกสารขนาดใหญ่เป็นส่วนย่อย (chunks) ที่จัดการได้ เพื่อให้พอดีกับหน้าต่างบริบทของ LLM และการค้นหาที่มีประสิทธิภาพ.[3]	Text Splitters (เช่น RecursiveCharacterTextSplitter) [11]
	Embedding (การสร้าง Embeddings)	การแปลงส่วนย่อยข้อความเป็นตัวแทนเวกเตอร์เชิงตัวเลขที่จับความหมายทางความหมาย.[3]	Embedding Models (เช่น gte-large) [12]
	Storing/Indexing (การจัดเก็บ/ทำดัชนี)	การจัดเก็บ embeddings ในฐานข้อมูลเวกเตอร์เพื่อการค้นหาความ	Vector Databases/Stores (เช่น MongoDB Atlas, Pinecone,

		คล้ายคลึงกันอย่างรวดเร็ว.[3]	Weaviate, Elasticsearch, Chroma, FAISS, Qdrant, Milvus)[3]
Runtime Query Processing (การประมวลผลคำถาม ณ รันไทม์)	Retrieval (การดึงข้อมูล)	การค้นหาส่วนย่อยของเอกสารที่เกี่ยวข้องมากที่สุดจากฐานข้อมูลเวกเตอร์ โดยอิงตามคำถามของผู้ใช้ที่ถูกแปลงเป็น embedding.[3]	Retriever (ส่วนหนึ่งของระบบ RAG), Vector Search [3]
	Augmentation (การเสริมข้อมูล)	การรวมข้อมูลที่ดึงมากับคำถามต้นฉบับของผู้ใช้ เพื่อสร้าง prompt ที่มีบริบทมากขึ้นสำหรับ LLM.[3]	RAG System's Integration Layer [5]
	Generation (การสร้างข้อความ)	LLM สร้างการตอบสนองสุดท้าย โดยใช้ prompt ที่ถูกเสริม ซึ่งมีข้อมูลที่ดึงมาเป็นบริบทเพิ่มเติม.[3]	Large Language Model (LLM) (เช่น GPT, Claude, Llama)[5]

เทคนิคและกลยุทธ์เพื่อประสิทธิภาพสูงสุดของ RAG

ส่วนนี้จะสำรวจวิธีการต่างๆ เพื่อเพิ่มประสิทธิภาพของระบบ RAG โดยเน้นที่การเพิ่มประสิทธิภาพทั้งส่วนประกอบการดึงข้อมูลและการสร้างข้อความ

รวมถึงการแนะนำสถาปัตยกรรม RAG ขั้นสูง

การปรับปรุง Retriever

การปรับปรุงประสิทธิภาพของตัวดึงข้อมูลเป็นสิ่งสำคัญในการรับรองว่าข้อมูลที่เกี่ยวข้องที่สุดจะถูกนำเสนอต่อ LLM

- **กลยุทธ์การดึงข้อมูลขั้นสูง (Advanced Retrieval Strategies)**
  - **Query Expansion:** เทคนิคนี้ใช้โมเดลเช่น Flan-T5 เพื่อขยายคำถามของผู้ใช้เดิมโดยการสร้างวลีคำหลักที่เกี่ยวข้อง.[16] การดำเนินการนี้จะขยายขอบเขตการค้นหาและสามารถปรับปรุงความถูกต้องของข้อเท็จจริงได้.[16]
  - **Chain-of-Retrieval Augmented Generation (CoRAG):** วิธีการนี้ช่วยให้โมเดลสามารถปรับปรุงคำถามแบบไดนามิกตามสถานะที่เปลี่ยนแปลงไป โดยดำเนินการดึงข้อมูลหลายขั้นตอน.[1] สิ่งนี้มีประสิทธิภาพอย่างยิ่งสำหรับงานตอบคำถามที่ซับซ้อนและต้องอาศัยการค้นหาหลายขั้นตอน.[1]
  - **Focus Mode Retrieval:** แทนที่จะดึงเอกสารทั้งหมด วิธีนี้จะดึงเฉพาะประโยคที่สำคัญที่สุดจากเอกสารที่ดึงมาเท่านั้น โดยรักษาสมดุลระหว่างบริบทที่กำหนดเป้าหมายกับการดึงข้อมูลที่ครอบคลุม.[16]

การเปลี่ยนแปลงจากวิธีการจับคู่คำหลักแบบง่ายไปสู่กลยุทธ์การดึงข้อมูลขั้นสูง เช่น การขยายคำถามและการดึงข้อมูลแบบหลายขั้นตอน (CoRAG) [1] แสดงให้เห็นถึงแนวโน้มที่ชัดเจนในการทำให้ระบบ RAG มีความชาญฉลาดและปรับตัวได้มากขึ้นในการค้นหาข้อมูล นี่คือการตอบสนองโดยตรงต่อข้อจำกัดของการดึงข้อมูลแบบครั้งเดียวแบบพื้นฐาน ซึ่งประสบปัญหาในการจัดการกับคำถามที่ซับซ้อนหรือกำกวม.[1] ผลกระทบที่กว้างขึ้นคือส่วนประกอบ "การดึงข้อมูล" ของ RAG กำลังมีความซับซ้อนมากขึ้น โดยก้าวข้ามการค้นหาความคล้ายคลึงกันแบบง่ายๆ ไปสู่การให้เหตุผล การวางแผน และการปรับปรุงซ้ำๆ ซึ่งทำให้เส้นแบ่งระหว่างการดึงข้อมูลบริสุทธิ์และตัวแทน AI อัจฉริยะเริ่มเลือนลางลง

- **เทคนิคการทำ Indexing (Indexing Techniques)**

    - **Vectorization:** การแปลงข้อความเป็นตัวแทนเวกเตอร์เชิงตัวเลข (embeddings) เป็นพื้นฐาน.[3]
    - **Inverted Index Matching (QuIM-RAG):** เป็นแนวทางใหม่ที่สร้างคำถามที่เป็นไปได้จากส่วนย่อยของเอกสาร และจับคู่คำถามเหล่านี้กับคำถามของผู้ใช้เพื่อระบุส่วนย่อยของข้อความที่เกี่ยวข้อง.[10] วิธีนี้สามารถใช้ดัชนี embedding แบบ quantized เพื่อลดภาระการคำนวณ.[10]
    - **Optimizing Document Indexing:** กลยุทธ์มุ่งเน้นไปที่การลดเวลาแฝงโดยไม่ลดทอนความแม่นยำ.[22]

ประสิทธิภาพของการดึงข้อมูลมีความเชื่อมโยงอย่างใกล้ชิดกับคุณภาพและการจัดระเบียบของข้อมูลที่ทำดัชนีไว้ หากกระบวนการทำดัชนี (การแบ่งส่วน, การสร้าง embedding, การจัดเก็บ) มีข้อบกพร่อง แม้แต่อัลกอริทึมการดึงข้อมูลที่ทันสมัยที่สุดก็ยังประสบปัญหาในการค้นหาข้อมูลที่เกี่ยวข้อง ตัวอย่างเช่น หากส่วนย่อยมีขนาดใหญ่เกินไป อาจทำให้ความเกี่ยวข้องลดลง หากเล็กเกินไป อาจทำให้บริบทแตกแยก.[16] QuIM-RAG [10] พยายามเพิ่มประสิทธิภาพในจุดนี้โดยการทำดัชนี *คำถาม* แทนที่จะเป็นเพียงเนื้อหา ซึ่งเป็นวิธีใหม่ในการปรับปรุงการจับคู่ความหมายระหว่างการดึงข้อมูล สิ่งนี้แสดงให้เห็นว่าขั้นตอนการเตรียมข้อมูล "ออฟไลน์" ในช่วงเริ่มต้นเป็นจุดสำคัญในการเพิ่มประสิทธิภาพ RAG โดยรวม

- **การใช้ Reranker (Leveraging Rerankers)**
    - **หน้าที่:** Rerankers เลือกเอกสารที่เกี่ยวข้องมากที่สุดจากกลุ่มเอกสารที่ดึงมา โดยทำการเลือกอย่างละเอียดด้วยความเข้าใจบริบทที่สมบูรณ์ยิ่งขึ้น.[23] สิ่งนี้ช่วยปรับปรุงคุณภาพของการตอบสนองที่สร้างขึ้นได้อย่างมาก.[23]
    - **HyperRAG:** เป็นระบบที่ปรับปรุงสมดุลระหว่างคุณภาพและประสิทธิภาพโดยใช้ประโยชน์จากการนำ KV-cache กลับมาใช้ใหม่เพื่อการอนุมาน reranker ที่มีประสิทธิภาพ.[23] โดยจะจัดเก็บ KV-cache ที่คำนวณไว้ล่วงหน้าของส่วนย่อยเอกสาร ซึ่งช่วยลดการคำนวณซ้ำและเวลาแฝง.[23]

- **ค่าใช้จ่ายในการคำนวณ:** แม้จะมีประสิทธิภาพ แต่ rerankers โดยเฉพาะอย่างยิ่งที่ใช้ LLM จะเพิ่มค่าใช้จ่ายในการคำนวณและเวลาแฝงเพิ่มเติม.[23]

Rerankers ให้การกรองความเกี่ยวข้อง "รอบที่สอง" ซึ่งช่วยปรับปรุงคุณภาพโดยการรับรองว่าข้อมูลที่เกี่ยวข้องที่สุดเท่านั้นจะถูกส่งไปยัง LLM.[23] อย่างไรก็ตาม สิ่งนี้ก็มีค่าใช้จ่ายเพิ่มเติมในด้านการคำนวณและเวลาแฝง.[23] HyperRAG [23] เป็นตัวอย่างของแนวคิดที่กำลังเกิดขึ้นในการวิจัย RAG: การเพิ่มประสิทธิภาพทั้งคุณภาพและประสิทธิภาพ การแลกเปลี่ยนที่ชัดเจนคือคุณภาพที่สูงขึ้นมักจะหมายถึงความต้องการในการคำนวณที่สูงขึ้น แนวคิดที่กำลังเกิดขึ้นคือการพัฒนาโซลูชันทางวิศวกรรมที่ชาญฉลาด (เช่น การนำ KV-cache กลับมาใช้ใหม่) เพื่อลดการแลกเปลี่ยนเหล่านี้ ทำให้ RAG คุณภาพสูงใช้งานได้จริงมากขึ้นสำหรับสภาพแวดล้อมการผลิต

การปรับปรุง Generator และ Prompt Engineering

การปรับปรุงส่วนประกอบ Generator และการออกแบบ Prompt อย่างมีประสิทธิภาพเป็นสิ่งสำคัญในการสร้างการตอบสนองที่แม่นยำและมีคุณภาพสูง

- **การเลือก LLM สำหรับ RAG (Choosing LLMs for RAG)**
  - LLMs ต่างๆ สามารถทำหน้าที่เป็นตัวสร้างได้ (เช่น GPT-3.5, GPT-4, Llama2, Falcon, PaLM, BERT).[7]
  - กรณีศึกษาแสดงให้เห็นว่า RAG ช่วยเพิ่มความแม่นยำของ LLMs บางตัวได้อย่างมาก (เช่น GPT-4.0 เพิ่มความแม่นยำ 11.3% ด้วย RAG) และบางครั้งมีประสิทธิภาพเหนือกว่าการตอบสนองที่สร้างโดยมนุษย์.[25] อย่างไรก็ตาม ประสิทธิภาพของ RAG อาจแตกต่างกันอย่างมากใน LLMs ที่แตกต่างกัน ตัวอย่างเช่น ความแม่นยำของ LLAMA2-13B ลดลงอย่างน่าประหลาดใจเมื่อใช้ RAG ในการศึกษาหนึ่ง.[25]

ข้อมูลที่นำเสนอมีความขัดแย้งกัน: RAG ช่วยเพิ่มความแม่นยำของ GPT-4.0 ได้อย่างมาก แต่กลับลดความแม่นยำของ LLAMA2-13B.[25] สิ่ง

นี้เน้นย้ำถึงความแตกต่างที่สำคัญ: RAG ไม่ใช่ยาครอบจักรวาลที่จะปรับปรุง LLM *ใดๆ* โดยอัตโนมัติ "วิธีการที่มีประสิทธิภาพดีที่สุด" สำหรับงานเฉพาะเจาะจงขึ้นอยู่กับ LLM ที่เลือกและคุณลักษณะโดยธรรมชาติของมัน (เช่น ความทนทานต่อบริบทที่มีสัญญาณรบกวน ความสามารถในการให้เหตุผล) สิ่งนี้บ่งชี้ว่านักวิจัยต้องทำการเปรียบเทียบเชิงประจักษ์กับ LLM และชุดข้อมูลเฉพาะของตนเพื่อกำหนดประสิทธิภาพที่เหมาะสมที่สุด แทนที่จะสันนิษฐานถึงความสามารถในการนำไปใช้ได้ทั่วไป

- **แนวทางการออกแบบ Prompt ที่มีประสิทธิภาพ (Effective Prompt Design Guidelines)**
    - **ความเฉพาะเจาะจงและความชัดเจน:** Prompt ต้องชัดเจนและเฉพาะเจาะจงเพื่อให้ได้การตอบสนองที่แม่นยำ โดยกำหนดงาน บริบท และผลลัพธ์ที่ต้องการ.[26]
    - **โครงสร้างข้อมูลนำเข้าและส่งออก:** จัดระเบียบข้อมูลนำเข้า (เช่น JSON, XML) และระบุรูปแบบข้อมูลส่งออก (เช่น รายการ, ย่อหน้า, โค้ด).[26]
    - **การใช้ Delimiters:** ใช้ตัวอักษรพิเศษหรือการจัดรูปแบบเพื่อแยกองค์ประกอบใน prompt เพื่อความชัดเจนและช่วยให้โมเดลจัดลำดับความสำคัญของส่วนสำคัญ.[26]
    - **การอธิบาย "ทำไม":** การให้เหตุผลเบื้องหลังคำแนะนำ (เช่น ทำไมสรุปควรจะกระชับ) สามารถปรับปรุงการปฏิบัติตามได้อย่างมาก.[27]
    - **การจัดการ "ฉันไม่รู้":** แทนที่จะสั่งว่า "อย่าหลอนข้อมูล" ให้เชื่อมโยงการตอบสนองที่ต้องการเข้ากับความเกี่ยวข้องของงาน เช่น "หากเนื้อหาไม่เพียงพอ ให้ระบุว่าเนื้อหาไม่เพียงพอที่จะให้คำตอบที่ดี".[27] สิ่งนี้ช่วยป้องกันการหลอนข้อมูล.[9]
    - **การเปรียบเทียบกับครู-นักเรียน:** ลองจินตนาการว่ากำลังให้คำแนะนำแก่นักเรียนสำหรับการทดสอบการอ่านเพื่อความเข้าใจ โดยเน้นว่าระบบควรตอบตามเอกสารที่ให้มาเท่านั้น.[27]
- **เทคนิค Prompt Engineering ขั้นสูง (Advanced Prompt**

Engineering Techniques)

- **Few-shot Prompting:** การให้ตัวอย่างคู่ข้อมูลนำเข้า-ส่งออกที่ต้องการไม่กี่ตัวอย่างเพื่อชี้นำรูปแบบ รูปแบบ และบริบทของโมเดล.[26]
- **Chain-of-Thought (CoT) Prompting:** กระตุ้นให้ LLM วิเคราะห์การตอบสนองทีละขั้นตอน ซึ่งช่วยปรับปรุงความโปร่งใส การวิเคราะห์ข้อผิดพลาด และความแม่นยำสำหรับคำถามที่ซับซ้อน.[21] ซึ่งอาจเกี่ยวข้องกับการระบุส่วนย่อยที่เกี่ยวข้อง การติดแท็ก การสรุป และการรวมสรุปในลักษณะ CoT.[21]
- **Prompt Chaining:** การแบ่งคำถามที่ซับซ้อนออกเป็นขั้นตอนย่อยที่จัดการได้ ซึ่ง prompt หลายตัวจะสร้างขึ้นจากกันและกันเพื่อการตอบสนองสุดท้ายที่แม่นยำยิ่งขึ้น.[26]
- **Emulating RAG via Prompt Engineering:** การระบุส่วนที่เกี่ยวข้องภายในข้อความยาวๆ การติดแท็ก การสรุป/วิเคราะห์ในลักษณะ CoT และจากนั้นสร้างคำตอบสุดท้ายโดยอ้างอิงหลักฐานที่ติดแท็กไว้.[21] สิ่งนี้สามารถใช้เป็นทางเลือกที่เบาสำหรับไปป์ไลน์การดึงข้อมูลแบบดั้งเดิมสำหรับการทำความเข้าใจบริบทที่ยาว.[21]

แนวทางปฏิบัติในการออกแบบ prompt ที่ละเอียด [21] แสดงให้เห็นว่าขั้นตอน "การเสริมข้อมูล" (การรวมบริบทที่ดึงมากับคำถาม) ไม่ใช่แค่การเชื่อมต่อข้อมูลเท่านั้น แต่เป็นศิลปะที่ซับซ้อน ความสัมพันธ์เชิงสาเหตุคือวิธีการนำเสนอข้อมูลต่อ LLM (การออกแบบ prompt) ส่งผลโดยตรงต่อความสามารถในการใช้บริบทที่ดึงมาได้อย่างมีประสิทธิภาพและสร้างคำตอบที่มีคุณภาพสูง แม้จะมีการดึงข้อมูลที่สมบูรณ์แบบ แต่ prompt ที่สร้างขึ้นไม่ดีก็อาจนำไปสู่ผลลัพธ์ที่ไม่เหมาะสมได้.[29] สิ่งนี้บ่งชี้ว่า prompt engineering เป็นการปรับปรุงประสิทธิภาพ RAG ใน "ขั้นตอนสุดท้าย" ที่สำคัญ และไม่ได้เกี่ยวกับสิ่งที่คุณดึงมาเท่านั้น แต่ยังเกี่ยวกับ *วิธีการที่คุณขอให้ LLM ใช้มัน* ด้วย

- **RAG ขั้นสูง: Agentic RAG และ GraphRAG (Advanced**

RAG: Agentic RAG and GraphRAG)

- **Agentic RAG (Agentic Retrieval-Augmented Generation):**
  - **แนวคิด:** ปรับปรุง RAG แบบดั้งเดิมโดยการรวมเอเจนต์ AI ที่เป็นอิสระเข้ากับไปป์ไลน์.[30] เอเจนต์เหล่านี้สามารถตัดสินใจแบบเรียลไทม์ วางแผนการกระทำ และประสานงานกับเครื่องมือหรือเอเจนต์อื่นๆ.[31] พวกมันไม่ปฏิบัติตามขั้นตอนการทำงานที่ตายตัว แต่จะประเมินความต้องการแบบไดนามิก ปรับปรุงบริบท และจัดลำดับขั้นตอน.[31]
  - **หลักการ:** ใช้ประโยชน์จากรูปแบบการออกแบบของเอเจนต์ เช่น การสะท้อน (การประเมินตนเอง), การวางแผน (การแบ่งงานย่อย), การใช้เครื่องมือ (ทรัพยากรภายนอก) และการทำงานร่วมกันของหลายเอเจนต์.[30]
  - **สถาปัตยกรรม:** สามารถเป็นแบบเอเจนต์เดียว, หลายเอเจนต์, แบบลำดับชั้น, หรือแบบแก้ไขข้อผิดพลาดได้.[32]
  - **จุดแข็ง:** การตอบสนองที่ชาญฉลาดและเกี่ยวข้องมากขึ้น, สร้างขึ้นเพื่อความซับซ้อน, ปรับขนาดได้และเป็นโมดูลาร์, อ้างอิงจากความรู้สด.[31] มีประสิทธิภาพในงานการให้เหตุผลแบบหลายขั้นตอนและการจัดการข้อมูลที่หลากหลายพร้อมกัน.[33]
  - **จุดอ่อน:** ภาระการคำนวณที่เพิ่มขึ้นและความซับซ้อนเนื่องจากการประสานงานของหลายเอเจนต์.[33]
- **GraphRAG:**
  - **แนวคิด:** ใช้ประโยชน์จากโครงสร้างข้อมูลแบบกราฟ (knowledge graphs) เพื่อเพิ่มประสิทธิภาพการดึงข้อมูล.[19] ในระหว่างการทำดัชนี LLMs จะแปลงส่วนย่อยของเอกสารให้เป็น knowledge graphs.[33]
  - **กลไก:** ใช้วิธีการค้นหาต่างๆ (การค้นหา node embedding, BM25, BFS) บนโหนดและขอบกราฟเพื่อ

การดึงข้อมูลที่แม่นยำสูง โดยเฉพาะอย่างยิ่งสำหรับคำถามที่มีโครงสร้างและการทำความเข้าใจความสัมพันธ์ระหว่างเอนทิตี.[19]

- **จุดแข็ง:** มีประสิทธิภาพสูงสำหรับคำถามที่มีโครงสร้างและสถานการณ์ที่การทำความเข้าใจความสัมพันธ์ระหว่างเอนทิตีมีความสำคัญอย่างยิ่ง.[33] ปรับปรุงความแม่นยำในการดึงข้อมูลโดยลดผลกระทบจากการแบ่งส่วนข้อความ.[34]
- **จุดอ่อน:** การพึ่งพาโครงสร้างกราฟอาจจำกัดความสามารถในการปรับขนาดกับชุดข้อมูลขนาดใหญ่หรือไม่เป็นโครงสร้าง; ค่าใช้จ่ายในการคำนวณในการบำรุงรักษาฐานข้อมูลกราฟ.[33]

การเกิดขึ้นของ Agentic RAG และ GraphRAG [19] แสดงให้เห็นถึงแนวโน้มที่สำคัญในการพัฒนา RAG: การก้าวข้าม "การดึงข้อมูลและสร้างข้อความ" แบบง่ายๆ ไปสู่ระบบที่ซับซ้อน ชาญฉลาด และปรับตัวได้มากขึ้น Agentic RAG นำเสนอการตัดสินใจแบบไดนามิกและการให้เหตุผลแบบหลายขั้นตอน ในขณะที่ GraphRAG ใช้ประโยชน์จากความรู้ที่มีโครงสร้างสำหรับการสอบถามความสัมพันธ์ที่แม่นยำ ผลกระทบที่กว้างขึ้นคือ RAG กำลังพัฒนาไปสู่กระบวนทัศน์ "ผู้ช่วย AI" ที่มีความสามารถในการแก้ปัญหาที่ซับซ้อน ไม่ใช่แค่การดึงข้อมูลเท่านั้น โดยการรวมการวางแผน การให้เหตุผล และการใช้เครื่องมือ สิ่งนี้ชี้ให้เห็นถึงอนาคตของแอปพลิเคชัน AI ที่มีลักษณะเป็นโมดูลาร์ สามารถประกอบเข้าด้วยกันได้สูง และรับรู้บริบท

ตาราง 2: การเปรียบเทียบวิธีการ RAG (Simple, Hybrid, Agentic, Graph)

วิธีการ RAG	คำอธิบาย/แนวคิด	จุดแข็ง	จุดอ่อน	กรณีการใช้งานที่เหมาะสม
Simple RAG	วิธีการพื้นฐาน	ง่ายต่อการนำ	ประสบปัญหา	คำถามที่พบ

	ที่รวมโมดูลดึงข้อมูลเดียวกับโมเดล generative.[33]	ไปใช้งานและต้องการทรัพยากรการคำนวณน้อยที่สุด.[33]	ในการจัดการกับความต้องการข้อมูลที่ซับซ้อนโดยเฉพาะคำถามที่กำกวมหรือมีหลายแง่มุม.[33]	บ่อย (FAQs) และการดึงความรู้ที่ไม่ซับซ้อน.[33]
Hybrid RAG	ปรับปรุงกระบวนการดึงข้อมูลโดยการรวมเทคนิคการดึงข้อมูลหลายวิธี (เช่น sparse และ dense retrieval).[33]	ปรับปรุงความแม่นยำในการดึงข้อมูลได้อย่างมากและแข็งแกร่งสำหรับสภาพแวดล้อมข้อมูลที่หลากหลาย.[33]	เพิ่มค่าใช้จ่ายในการคำนวณและมีความซับซ้อนในการตั้งค่าและการบำรุงรักษา.[33]	สถานการณ์การดึงข้อมูลที่ซับซ้อนซึ่งเกี่ยวข้องกับเอกสารที่มีรูปแบบและแหล่งที่มาที่หลากหลาย.[33]
AgenticRAG	ใช้สถาปัตยกรรม multi-agent ที่เอเจนต์ต่างๆ เชี่ยวชาญในงานดึงข้อมูลหรือสร้างข้อความที่แตกต่างกัน.[31]	มีความสามารถในการปรับตัวสูงและมีประสิทธิภาพในสถานการณ์ที่ต้องจัดการกับข้อมูลหลายประเภทพร้อมกัน.[31]	เพิ่มภาระการคำนวณและความซับซ้อนในการประสานงานของหลายเอเจนต์.[33]	การวิเคราะห์อนุกรมเวลาและงานการให้เหตุผลแบบหลายขั้นตอนที่ซับซ้อนซึ่งต้องการการประมวลผลข้อมูลที่หลากหลาย.[33]
GraphRAG	ใช้ประโยชน์จากโครงสร้าง	มีประสิทธิภาพสูงสำหรับคำ	การพึ่งพาโครงสร้าง	การสอบถาม knowledge

	ข้อมูลแบบกราฟเพื่อเพิ่มประสิทธิภาพการดึงข้อมูล.[19]	ถามที่มีโครงสร้างและสถานการณ์ที่การทำความเข้าใจความสัมพันธ์ระหว่างเอนทิตีมีความสำคัญ.[33]	กราฟอาจจำกัดความสามารถในการปรับขนาดกับชุดข้อมูลขนาดใหญ่หรือไม่เป็นโครงสร้าง.[33]	graph และงานดึงข้อมูลเฉพาะโดเมนที่การทำความเข้าใจความสัมพันธ์ของเอนทิตีมีความสำคัญ.[33]

การวัดผลและประเมินประสิทธิภาพของ RAG

ส่วนนี้จะครอบคลุมประเด็นสำคัญของการประเมินระบบ RAG โดยนำเสนอวิธีการและเมตริกเพื่อกำหนดประสิทธิภาพของแนวทางต่างๆ

ความสำคัญของการประเมิน RAG

การประเมินเป็นสิ่งสำคัญในการพิจารณาคุณภาพ ต้นทุน และเวลาแฝงของแอปพลิเคชัน RAG และเพื่อให้แน่ใจว่าเป็นไปตามข้อกำหนดทางธุรกิจ.[13] ระบบ RAG ที่ไม่ได้รับการประเมินอย่างละเอียดอาจนำไปสู่ "ความล้มเหลวแบบเงียบ" ซึ่งบ่อนทำลายความน่าเชื่อถือของระบบโดยรวม.[17] การประเมินอย่างเข้มงวดช่วยให้เข้าใจการทำงานของไปป์ไลน์ที่ซับซ้อนและหลายขั้นตอน และระบุช่องว่างด้านประสิทธิภาพ.[35] นอกจากนี้ คำอธิบายที่โปร่งใสสำหรับการตัดสินใจของ AI (เช่น RAGXplain) ยังส่งเสริมความไว้วางใจของผู้ใช้ ซึ่งเป็นความท้าทายสำคัญในการนำ AI มาใช้.[35]

กรอบการทดสอบและเมตริกการประเมิน

- **การสร้างกรอบการทดสอบ:**
  - **ชุดข้อมูลทดสอบ:** สร้างชุดข้อมูลทดสอบของคำถามคุณภาพสูงที่ครอบคลุมชุดย่อยที่กว้างขวางของข้อมูลพื้นฐานและสะท้อนกรณีการใช้งานจริง รวมถึงความหลากหลายในการใช้ถ้อยคำและความซับซ้อน.[17]
  - **ชุดข้อมูลอ้างอิง 'Golden':** รวบรวมชุดข้อมูลของผลลัพธ์ที่

ต้องการสำหรับการประเมินที่ครอบคลุม.[17]
    - **การทดลองควบคุม:** เปลี่ยนตัวแปรทีละตัวเท่านั้นระหว่างการทดสอบ เพื่อให้สามารถระบุการเปลี่ยนแปลงประสิทธิภาพได้อย่างมั่นใจกับคุณลักษณะเฉพาะนั้น.[17] หลีกเลี่ยงการเปลี่ยนแปลงคำถามประเมินหรือพารามิเตอร์ของระบบระหว่างการทดสอบ.[17]
    - **การวิเคราะห์หาสาเหตุของปัญหา (Root Cause Analysis):** แยกปัญหาไปยังความแม่นยำในการดึงข้อมูลหรือการประมวลผลบริบทของ LLM เพื่อระบุปัญหา.[17]
- **เมตริกเชิงปริมาณและเชิงคุณภาพ (Quantitative and Qualitative Metrics)**
    - **เมตริกเชิงปริมาณ:**
        - **เมตริกที่ใช้ LLM:** ให้ความสำคัญเนื่องจากความเข้าใจทางความหมายที่เหนือกว่าและความสามารถในการสร้างคำอธิบายที่มนุษย์อ่านได้.[36]
            - **Pointwise Metrics:** โมเดล Judge กำหนดคะแนนตัวเลข (เช่น 0-5) ให้กับการจัดเรียงผลลัพธ์ตามเกณฑ์.[17]
            - **Pairwise Metrics:** โมเดล Judge เปรียบเทียบการตอบสนองของสองโมเดลเพื่อระบุโมเดลที่เหนือกว่า.[17]
        - **เมตริกที่ใช้การคำนวณ:** ใช้สูตรทางคณิตศาสตร์เพื่อเปรียบเทียบผลลัพธ์กับ ground truth (เช่น ROUGE, BLEU).[17]
        - **เมตริกเฉพาะ RAG (RAGAS):** ความเกี่ยวข้องของบริบท, ความน่าเชื่อถือ, ความเกี่ยวข้องของคำตอบ, ความแม่นยำของบริบท, การเรียกคืนบริบท.[17]
        - **เมตริกอื่นๆ:** การอ้างอิงการตอบสนอง, ความละเอียด, การปฏิบัติตามคำแนะนำ, คุณภาพคำตอบของคำถาม.[17]
    - **เมตริกเชิงคุณภาพ:** การตัดสินของมนุษย์มีความสำคัญอย่าง

ยิ่งสำหรับลักษณะที่ละเอียดอ่อน เช่น น้ำเสียง ความชัดเจน และความกำกวม ซึ่งเครื่องมืออัตโนมัติไม่สามารถทำซ้ำได้อย่างสมบูรณ์.[17]

การใช้ Judge Models และ Root Cause Analysis

- **Judge Models:** LLMs ที่เป็นกรรมสิทธิ์หรือได้รับการปรับแต่งสามารถทำหน้าที่เป็นผู้ประเมิน โดยให้คะแนนการตอบสนองตามเกณฑ์ที่กำหนดไว้ล่วงหน้า.[17]
- **RAGXplain:** เป็นกรอบการประเมินที่วัดประสิทธิภาพ RAG และแปลการประเมินเป็นข้อมูลที่ชัดเจนและคำแนะนำที่นำไปปฏิบัติได้โดยใช้การให้เหตุผลของ LLM.[35] โดยมีโมดูลคำอธิบายหลายขั้นตอนสำหรับการดึงข้อมูลและขั้นตอนการสร้างข้อความ.[35]
- **Root Cause Analysis:** การระบุและแยกแยะความแม่นยำในการดึงข้อมูลและการประมวลผลบริบทของ LLM ช่วยให้สามารถปรับปรุงประสิทธิภาพได้อย่างตรงจุด.[17] ตัวอย่างเช่น การทดสอบจำนวนเพื่อนบ้านที่เหมาะสม, การเลือกโมเดล embedding, กลยุทธ์การแบ่งส่วน และการประมวลผลล่วงหน้าของส่วนย่อย.[17]

การอภิปรายอย่างกว้างขวางเกี่ยวกับกรอบการประเมินและเมตริก [13] ย้ำว่าการเพิ่มประสิทธิภาพ RAG เป็นวิทยาศาสตร์เชิงประจักษ์ ไม่ใช่ระบบที่ "ตั้งค่าแล้วลืม" ผลกระทบที่กว้างขึ้นสำหรับนักวิจัยคือการประเมินอย่างเข้มงวดและเป็นระบบมีความสำคัญสูงสุดในการระบุ "วิธีการที่มีประสิทธิภาพดีที่สุด" ความสัมพันธ์เชิงสาเหตุคือหากไม่มีกรอบการทดสอบที่ชัดเจน (การทดลองควบคุม, เมตริกที่กำหนด, การวิเคราะห์หาสาเหตุของปัญหา) ก็ไม่สามารถระบุได้อย่างแน่ชัดว่า *ทำไม* การเปลี่ยนแปลงบางอย่างจึงปรับปรุงหรือลดประสิทธิภาพ ซึ่งทำให้การเพิ่มประสิทธิภาพที่แท้จริงเป็นไปได้ยาก RAGXplain ที่เน้น "คำแนะนำที่นำไปปฏิบัติได้" [35] ตอบสนองความต้องการนี้โดยตรงสำหรับข้อเสนอแนะในการวินิจฉัยเชิงปฏิบัติ

การประเมินโดยมนุษย์ (Human Evaluation)

- **บทบาท:** ให้ข้อเสนอแนะเชิงคุณภาพที่สำคัญซึ่งเครื่องมืออัตโนมัติไม่สามารถทำซ้ำได้.[17]
- **กระบวนการ:** โดยทั่วไปจะดำเนินการหลังจากบรรลุคุณภาพคำตอบพื้นฐานที่แข็งแกร่งผ่านการทดสอบอัตโนมัติ เกี่ยวข้องกับการสรรหาผู้ใช้ที่เป็นตัวแทน (personas) เพื่อประเมินคุณลักษณะเฉพาะหรือประสิทธิภาพของแอปพลิเคชันโดยรวม.[17]
- **ความสำคัญ:** จำเป็นสำหรับการทำความเข้าใจลักษณะที่ละเอียดอ่อน เช่น น้ำเสียง ความชัดเจน และความกำกวม และประสบการณ์ผู้ใช้.[17]

ในขณะที่เมตริกเชิงปริมาณ (เช่น RAGAS) มีประสิทธิภาพ การเน้นการประเมินโดยมนุษย์ [17] เผยให้เห็นรูปแบบที่ซ่อนอยู่: ระบบ RAG แม้จะมีความซับซ้อนทางเทคนิค แต่ท้ายที่สุดแล้วก็ถูกออกแบบมาเพื่อการโต้ตอบกับมนุษย์ ดังนั้น คุณสมบัติเชิงอัตวิสัย เช่น "น้ำเสียง ความชัดเจน และความกำกวมที่อาจเกิดขึ้น" [17] จึงเป็นตัวบ่งชี้ประสิทธิภาพที่สำคัญซึ่งการตัดสินของมนุษย์เท่านั้นที่สามารถจับได้ สิ่งนี้บ่งชี้ว่าระบบ RAG ที่ "มีประสิทธิภาพดีที่สุด" อย่างแท้จริงจะต้องตอบสนองทั้งเมตริกทางเทคนิคเชิงวัตถุประสงค์และประสบการณ์ของมนุษย์เชิงอัตวิสัย ซึ่งเน้นย้ำถึงแง่มุมการออกแบบที่เน้นมนุษย์เป็นศูนย์กลางของ AI

ตาราง 3: เมตริกการประเมินหลักสำหรับระบบ RAG

หมวดหมู่เมตริก	ชื่อเมตริก	คำอธิบาย	เครื่องมือ/เฟรมเวิร์ก
Quantitative - LLM-based	Pointwise Score	โมเดล Judge กำหนดคะแนนตัวเลข (เช่น 0-5) ให้กับการจัดเรียงผลลัพธ์ตามเกณฑ์.[17]	Vertex AI Eval Service, RAGXplain [17]
	Pairwise Score	โมเดล Judge	Vertex AI Eval

		เปรียบเทียบการตอบสนองของสองโมเดลเพื่อระบุโมเดลที่เหนือกว่า.[17]	Service [17]
Quantitative - Computation-based	ROUGE	วัดความคล้ายคลึงกันของข้อความที่สร้างขึ้นกับข้อความอ้างอิง โดยคำนวณความทับซ้อนของ n-gram.[17]	- [17]
	BLEU	วัดความคล้ายคลึงกันของข้อความที่สร้างขึ้นกับข้อความอ้างอิง โดยคำนวณความแม่นยำของ n-gram.[17]	- [17]
RAG-Specific	Context Relevance	วัดว่าบริบทที่ดึงมามีความเกี่ยวข้องกับคำถามของผู้ใช้มากน้อยเพียงใด.[17]	RAGAS [17]
	Faithfulness	วัดว่าการตอบสนองที่สร้างขึ้นนั้นสอดคล้องกับ	RAGAS [17]

		ข้อเท็จจริงในบริบทที่ดึงมามากน้อยเพียงใด.[17]	
	Answer Relevance	วัดว่าคำตอบที่สร้างขึ้นมีความเกี่ยวข้องกับคำถามของผู้ใช้มากน้อยเพียงใด.[17]	RAGAS [17]
	Context Precision	วัดว่าเอกสารที่ดึงมามีความแม่นยำในการตอบคำถามหรือไม่.[19]	RAGAS [19]
	Context Recall	วัดว่าเอกสารที่ดึงมาครอบคลุมข้อมูลที่จำเป็นสำหรับการตอบคำถามได้ดีเพียงใด.[19]	RAGAS [19]
	Response Groundedness	วัดว่าข้อความที่สร้างขึ้นสอดคล้องกับข้อมูลข้อเท็จจริงจากเอกสารต้นฉบับมากน้อยเพียงใด.[17]	Vertex AI Eval Service [17]
	Verbosity	ความยาวและรายละเอียดของการตอบสนอง ซึ่งอาจต้องปรับแต่งตาม	Vertex AI Eval Service [17]

		กรณีการใช้งาน.[17]	
	Instruction Following	ความสามารถของระบบในการปฏิบัติตามคำแนะนำที่กำหนด เพื่อให้มั่นใจว่าผลลัพธ์มีความเกี่ยวข้องและสอดคล้อง.[17]	Vertex AI Eval Service [17]
	Question Answer Quality	ความสามารถของระบบ RAG ในการตอบคำถามของผู้ใช้ได้อย่างถูกต้อง พร้อมรายละเอียดและความสอดคล้อง.[17]	Vertex AI Eval Service [17]
Qualitative	Human Judgment	การประเมินโดยมนุษย์สำหรับลักษณะที่ละเอียดอ่อน เช่น น้ำเสียง ความชัดเจน และความกำกวม.[17]	Human Testers [17]

ความท้าทายและแนวทางแก้ไขใน RAG

ส่วนนี้จะกล่าวถึงความท้าทายและจุดล้มเหลวที่พบบ่อยในการนำระบบ RAG ไปใช้งาน พร้อมด้วยแนวทางแก้ไขที่เป็นประโยชน์และข้อควรพิจารณาสำหรับการพัฒนาระบบที่แข็งแกร่ง

ปัญหาที่พบบ่อยและจุดที่ระบบล้มเหลว

การทำความเข้าใจจุดล้มเหลวเป็นสิ่งสำคัญสำหรับการสร้างระบบ RAG ที่แข็งแกร่ง การวิเคราะห์ระบุจุดล้มเหลว 7 ประการที่อาจเกิดขึ้น: [37]

- **FP1: Missing Content (เนื้อหาขาดหายไป):** เกิดขึ้นเมื่อระบบ RAG ถูกถามคำถามที่ไม่มีคำตอบในเอกสารที่มีอยู่.[37] ในสถานการณ์ที่เหมาะสม ระบบควรตอบว่า "ขออภัย ไม่ทราบข้อมูล" แต่บางครั้ง RAG อาจ "หลอน" คำตอบสำหรับคำถามที่เกี่ยวข้องกับเนื้อหาแต่ไม่มีคำตอบโดยตรงในเอกสารที่ให้มา.[37] สิ่งนี้เน้นย้ำถึงความจำเป็นในการ "ตรวจจับข้อมูลนอกโดเมน".[9]
- **FP2: Missed the Top Ranked Documents (พลาดเอกสารที่จัดอันดับสูงสุด):** แม้ว่าคำตอบสำหรับคำถามจะมีอยู่ในเอกสาร แต่จุดล้มเหลวนี้เกิดขึ้นเมื่อเอกสารที่เกี่ยวข้องไม่ได้รับการจัดอันดับสูงพอที่จะรวมอยู่ในชุดเอกสารที่ส่งคืนให้ผู้ใช้.[37] ในทางปฏิบัติ ระบบ RAG มักจะดึงเอกสารเพียง 'K' อันดับแรกเท่านั้น (โดยที่ 'K' เป็นค่าที่อิงตามประสิทธิภาพ) ซึ่งหมายความว่าหากเอกสารสำคัญอยู่นอก 'K' อันดับแรก คำตอบจะถูกพลาดไป.[37]
- **FP3: Not in Context - Consolidation Strategy Limitations (ไม่อยู่ในบริบท - ข้อจำกัดของกลยุทธ์การรวม):** ความล้มเหลวนี้เกิดขึ้นเมื่อเอกสารที่มีคำตอบที่ถูกต้องถูกดึงมาจากฐานข้อมูลสำเร็จ แต่ไม่ถูกนำไปใช้ในบริบทสุดท้ายที่ส่งให้กับ LLM เพื่อสร้างคำตอบ.[37] สิ่งนี้มักเกิดขึ้นเมื่อมีการดึงเอกสารจำนวนมาก และมีการใช้กระบวนการรวมเพื่อเลือกชุดย่อยสำหรับ LLM โดยไม่ตั้งใจที่จะยกเว้นข้อมูลที่สำคัญ.[37]
- **FP4: Not Extracted (ไม่ถูกสกัด):** ในสถานการณ์นี้ คำตอบมีอยู่ในบริบทที่ให้กับ LLM แต่ LLM เองล้มเหลวในการสกัดคำตอบที่ถูกต้อง.[37] สิ่งนี้มักเกิดขึ้นเมื่อบริบทมี "สัญญาณรบกวน" มากเกินไป (ข้อมูลที่ไม่เกี่ยวข้อง) หรือข้อมูลที่ขัดแย้งกัน ทำให้ LLM ยากที่จะระบุและสังเคราะห์การตอบสนองที่แม่นยำ.[37]
- **FP5: Wrong Format (รูปแบบผิด):** จุดล้มเหลวนี้เกิดขึ้นเมื่อคำถามของผู้ใช้ระบุอย่างชัดเจนว่าต้องการข้อมูลในรูปแบบเฉพาะ (เช่น ตารางหรือรายการ) แต่ LLM ไม่สนใจคำแนะนำนี้และให้คำตอบในรูปแบบอื่น.[37]

- **FP6: Incorrect Specificity (ความเฉพาะเจาะจงไม่ถูกต้อง):** คำตอบที่ส่งคืนโดยระบบ RAG ไม่เฉพาะเจาะจงพอหรือไม่เฉพาะเจาะจงเกินไปที่จะตอบสนองความต้องการของผู้ใช้ได้อย่างเพียงพอ.[37] สิ่งนี้อาจเกิดขึ้นเมื่อผู้ออกแบบระบบ RAG มีผลลัพธ์ที่ต้องการสำหรับคำถามที่กำหนด (เช่น การให้เนื้อหาการศึกษาเฉพาะสำหรับนักเรียนมากกว่าแค่คำตอบโดยตรง) นอกจากนี้ยังเกิดขึ้นเมื่อผู้ใช้ถามคำถามที่กว้างเกินไปเนื่องจากไม่แน่ใจว่าจะใช้ถ้อยคำที่แม่นยำได้อย่างไร.[37]
- **FP7: Incomplete (ไม่สมบูรณ์):** คำตอบที่ไม่สมบูรณ์ไม่จำเป็นต้องไม่ถูกต้อง แต่ขาดข้อมูลสำคัญบางอย่างที่มีอยู่ในบริบทที่ให้มาและสามารถสกัดได้.[37]

จุดล้มเหลวทั้งเจ็ดประการ [37] แสดงให้เห็นถึงความล้มเหลวที่อาจเกิดขึ้นตลอดไปป์ไลน์ RAG ตั้งแต่การนำเข้าข้อมูลไปจนถึงการสร้างข้อความขั้นสุดท้าย ตัวอย่างเช่น "เนื้อหาขาดหายไป" (FP1) ชี้ให้เห็นถึงปัญหาในการครอบคลุมฐานความรู้ ในขณะที่ "ไม่อยู่ในบริบท" (FP3) และ "ไม่ถูกสกัด" (FP4) ชี้ให้เห็นถึงปัญหาในการจัดการหน้าต่างบริบทหรือความสามารถของ LLM ในการประมวลผล รูปแบบที่ซ่อนอยู่คือระบบ RAG มีความซับซ้อนโดยธรรมชาติ มีส่วนประกอบหลายส่วน และความล้มเหลวในส่วนหนึ่ง (เช่น การดึงข้อมูล) สามารถส่งผลกระทบต่อเนื่องและแสดงออกมาเป็นปัญหาที่ดูเหมือนจะเป็น "การสร้างข้อความ" (เช่น การหลอนข้อมูล) ทำให้การวิเคราะห์หาสาเหตุของปัญหาเป็นเรื่องที่ท้าทายหากไม่มีกรอบการประเมินที่เป็นระบบ

แนวทางแก้ไขและข้อควรพิจารณา

- **การนำเข้าและดึงข้อมูลที่มีความแม่นยำสูง:** ประมวลผลข้อมูลที่หลากหลายและไม่มีโครงสร้างโดยใช้เทคนิคที่มีความแม่นยำสูง การระบุแหล่งที่มาอย่างละเอียดช่วยให้มั่นใจในความถูกต้องและตรวจสอบได้.[9]
- **ความเข้าใจคำถาม:** ปรับให้เข้ากับศัพท์เฉพาะและคำย่อของอุตสาหกรรม แบ่งคำถามที่ซับซ้อนออกเป็นส่วนย่อย รักษาสภาพ

บริบทระหว่างเซสชัน และตรวจจับเจตนาของผู้ใช้.[2] การเขียนคำถามใหม่สามารถเพิ่มประสิทธิภาพการค้นหาได้.[2]

- **ลดการหลอนข้อมูล:** ใช้แนวทางปฏิบัติในการนำเข้าและดึงข้อมูลขั้นสูง และใช้การตรวจจับข้อมูลนอกโดเมนสำหรับคำถามที่ไม่สามารถตอบได้.[9] การอ้างอิงโมเดลกับบริบทที่ดึงมาช่วยเพิ่มความถูกต้องของข้อเท็จจริง.[4]
- **ประสิทธิภาพเทียบกับเวลาแฝงเทียบกับต้นทุน:** ทุกการตัดสินใจในระหว่างการนำเข้าข้อมูล การดึงข้อมูล และการสร้างข้อความส่งผลกระทบต่อปัจจัยเหล่านี้ การปรับปรุงประสิทธิภาพหนึ่งอย่างอาจส่งผลกระทบต่อสิ่งอื่น.[9]
- **ความปลอดภัยและการปฏิบัติตามข้อกำหนด:** รวมการเข้ารหัส การจัดเก็บข้อมูลที่ปลอดภัย การควบคุมการเข้าถึง (ระดับเอกสาร) SSO และการปฏิบัติตามมาตรฐานข้อกำหนด.[5]
- **การทดสอบและปรับปรุงซ้ำๆ:** การตรวจสอบระบบ RAG มักจะทำได้ในระหว่างการใช้งานเท่านั้น และความแข็งแกร่งของระบบจะพัฒนาขึ้นเรื่อยๆ แทนที่จะถูกออกแบบมาตั้งแต่ต้น.[37] สิ่งนี้ต้องการการตรวจสอบอย่างต่อเนื่องและการปรับปรุงซ้ำๆ.[17]

การ "สมดุลระหว่างประสิทธิภาพ เวลาแฝง และต้นทุน" [9] เป็นการแลกเปลี่ยนพื้นฐานในการออกแบบระบบ RAG ตัวอย่างเช่น การดึงเอกสารจำนวนมากขึ้นอาจปรับปรุงความแม่นยำแต่เพิ่มเวลาแฝงและต้นทุน การใช้ LLM ที่มีขนาดเล็กและเร็วกว่าอาจลดต้นทุนแต่ส่งผลกระทบต่อคุณภาพการสร้างข้อความ สิ่งนี้บ่งชี้ว่าไม่มีโซลูชัน "ดีที่สุด" เพียงอย่างเดียว แต่เป็นการกำหนดค่าที่เหมาะสมที่สุดสำหรับกรณีการใช้งานเฉพาะ ซึ่งต้องมีการปรับเทียบอย่างระมัดระวัง แนวทางปฏิบัติที่ดีที่สุดคือการกำหนดเป้าหมายประสิทธิภาพที่ชัดเจน (เช่น เวลาตอบสนอง, เกณฑ์ความแม่นยำ) ตั้งแต่เริ่มต้น และใช้การทดสอบแบบวนซ้ำเพื่อจัดการกับการแลกเปลี่ยนเหล่านี้อย่างมีประสิทธิภาพ

ตาราง 4: จุดที่ระบบ RAG ล้มเหลวและกลยุทธ์การแก้ไขที่เป็นไปได้

จุดที่ระบบล้มเหลว (Failure Point)	คำอธิบาย (Description)	กลยุทธ์การแก้ไขที่เป็นไปได้ (Potential Mitigation Strategies)
Missing Content (เนื้อหาขาดหายไป)	คำตอบสำหรับคำถามไม่มีอยู่ในเอกสารที่มีอยู่ ทำให้ระบบอาจหลอนข้อมูล.[37]	ขยายฐานความรู้; ใช้การตรวจจับข้อมูลนอกโดเมน (Out-of-Domain Detection) เพื่อแจ้งว่าไม่สามารถตอบคำถามได้.[9]
Missed the Top Ranked Documents (พลาดเอกสารที่จัดอันดับสูงสุด)	เอกสารที่เกี่ยวข้องมีอยู่ แต่ไม่ได้รับการจัดอันดับสูงพอที่จะถูกดึงมา.[37]	ปรับปรุงโมเดล Embedding; ใช้เทคนิค Query Expansion; ใช้ Reranker เพื่อปรับปรุงความแม่นยำในการดึงข้อมูล.[16]
Not in Context - Consolidation Strategy Limitations (ไม่อยู่ในบริบท - ข้อจำกัดของกลยุทธ์การรวม)	เอกสารที่เกี่ยวข้องถูกดึงมาแต่ไม่ถูกนำไปใช้ในบริบทสุดท้ายที่ส่งให้กับ LLM.[37]	ปรับปรุงกลยุทธ์การแบ่งส่วน (Chunking); ปรับปรุงกลยุทธ์การรวมบริบท; เพิ่มขนาดหน้าต่างบริบทของ LLM (หากเป็นไปได้).[16]
Not Extracted (ไม่ถูกสกัด)	คำตอบมีอยู่ในบริบทที่ให้กับ LLM แต่ LLM ไม่สามารถสกัดคำตอบที่ถูกต้องได้.[37]	ปรับปรุง Prompt Engineering (เช่น Chain-of-Thought); ลด "สัญญาณรบกวน" ในบริบท; Fine-tune LLM เพื่อการสกัดข้อมูลที่ดีขึ้น.[21]

Wrong Format (รูปแบบผิด)	LLM ไม่สนใจคำแนะนำสำหรับรูปแบบผลลัพธ์ที่ต้องการ.[37]	กำหนดรูปแบบผลลัพธ์ที่ชัดเจนใน Prompt (Structured Outputs); ใช้ Delimiters เพื่อแยกส่วน Prompt.[26]
Incorrect Specificity (ความเฉพาะเจาะจงไม่ถูกต้อง)	คำตอบที่ส่งคืนโดยระบบ RAG ไม่เฉพาะเจาะจงพอหรือไม่เฉพาะเจาะจงเกินไป.[37]	ปรับปรุงการทำความเข้าใจเจตนาของผู้ใช้; ใช้ Prompt Engineering เพื่อชี้นำความเฉพาะเจาะจงที่ต้องการ; ใช้ Agentic RAG สำหรับการให้เหตุผลที่ซับซ้อนขึ้น.[2]
Incomplete (ไม่สมบูรณ์)	คำตอบขาดข้อมูลสำคัญบางอย่างที่มีอยู่ในบริบท.[37]	ปรับปรุงกลยุทธ์การดึงข้อมูล (เช่น CoRAG สำหรับ multi-hop); เพิ่มจำนวนเอกสารที่ดึงมา; ปรับปรุง Prompt Engineering เพื่อให้ LLM ครอบคลุมข้อมูลทั้งหมด.[1]

เครื่องมือและเฟรมเวิร์ก Open-Source สำหรับ RAG

ส่วนนี้จะให้ภาพรวมของเฟรมเวิร์กและไลบรารีโอเพนซอร์สยอดนิยมที่ช่วยอำนวยความสะดวกในการพัฒนาและปรับใช้ระบบ RAG ซึ่งเป็นแนวทางในการเลือกเครื่องมือที่เหมาะสม

ภาพรวมเฟรมเวิร์กยอดนิยม

- **LangChain:** เฟรมเวิร์กยอดนิยมสำหรับการสร้างแอปพลิเคชัน LLM มีให้เลือกทั้งใน Python และ JavaScript มี Document Loader ที่

หลากหลาย, Text Splitter ที่มีประสิทธิภาพ, การรวม Vector Store (Chroma, Pinecone, FAISS, Weaviate), การสนับสนุน Embedding ที่ทรงพลัง, Retriever ขั้นสูง, Unified LLM Interface และการสนับสนุนหน่วยความจำในตัว.[11]

- **LlamaIndex:** เฟรมเวิร์กข้อมูลที่ครอบคลุมสำหรับการเชื่อมต่อ LLMs กับแหล่งข้อมูลส่วนตัว โดยให้แนวทางที่มีโครงสร้างสำหรับการนำเข้าข้อมูล, การทำดัชนี และการดึงข้อมูล.[3] มีตัวเชื่อมต่อข้อมูลที่ยืดหยุ่น, การทำดัชนีที่ปรับแต่งได้, กลไกการดึงข้อมูลขั้นสูง และการสนับสนุนหลายโมดอล.[19]
- **Dify:** แพลตฟอร์มพัฒนาแอปพลิเคชัน LLM แบบโอเพนซอร์สพร้อมการสร้างเวิร์กโฟลว์ด้วยภาพและความสามารถ RAG ที่ทรงพลัง เข้าถึงได้ทั้งนักพัฒนาและผู้ใช้ที่ไม่ใช่ด้านเทคนิค.[19]
- **RAGFlow:** เอ็นจิ้น RAG แบบโอเพนซอร์สที่เน้นการทำความเข้าใจเอกสารเชิงลึก มีความสามารถในการสกัดข้อมูลที่มีโครงสร้างจากเอกสารที่ซับซ้อน (PDFs, ตาราง, เค้าโครง).[15]
- **LangGraph:** สร้างขึ้นบน LangChain โดยให้โครงสร้างแบบกราฟสำหรับเวิร์กโฟลว์ RAG ที่ซับซ้อนและหลายขั้นตอน ทำให้สามารถใช้สถาปัตยกรรมแบบโหนด, การไหลแบบขับเคลื่อนด้วยขอบ และการสนับสนุนหลายเอเจนต์.[15]
- **Milvus:** ฐานข้อมูลเวกเตอร์ประสิทธิภาพสูงแบบคลาวด์เนทีฟที่สร้างขึ้นโดยเฉพาะสำหรับการค้นหาความคล้ายคลึงกันของเวกเตอร์ที่ปรับขนาดได้ ซึ่งมีความสำคัญต่อการจัดเก็บและดึงข้อมูล Embedding อย่างมีประสิทธิภาพ.[3]
- **mem0:** เลเยอร์หน่วยความจำอัจฉริยะที่ช่วยเพิ่มประสิทธิภาพ RAG ด้วยความสามารถในการเก็บหน่วยความจำตามบริบทอย่างถาวร ทำให้ระบบ AI สามารถเรียนรู้และปรับตัวจากการโต้ตอบของผู้ใช้เมื่อเวลาผ่านไป.[19]
- **DSPy:** เฟรมเวิร์กสำหรับการ "เขียนโปรแกรม" (มากกว่าการ "Prompt") โมเดลภาษา ทำให้สามารถสร้างระบบดึงข้อมูลแบบโมดู

ลาร์ที่ปรับปรุงตนเองได้ผ่านโค้ด Python แบบประกาศ.[19]

- **Haystack:** เฟรมเวิร์กการจัดเรียง AI แบบครบวงจรที่ออกแบบมาสำหรับการสร้างแอปพลิเคชัน LLM ที่ยืดหยุ่นและพร้อมใช้งานสำหรับการผลิตด้วยสถาปัตยกรรมส่วนประกอบแบบโมดูลาร์.[19]
- **RAGAS:** ชุดเครื่องมือประเมินที่ครอบคลุมซึ่งออกแบบมาโดยเฉพาะสำหรับการประเมินและเพิ่มประสิทธิภาพแอปพลิเคชัน RAG โดยให้เมตริกเชิงวัตถุประสงค์และการสร้างข้อมูลทดสอบ.[17]

ปริมาณและความเชี่ยวชาญของเฟรมเวิร์ก RAG แบบโอเพนซอร์ส [15] ที่มีอยู่มากมายบ่งชี้อย่างชัดเจนถึงระบบนิเวศที่กำลังเติบโตและมีความหลากหลาย แนวโน้มนี้หมายความว่าชุมชน RAG กำลังก้าวไปสู่โซลูชันที่เป็นโมดูลาร์ สามารถประกอบเข้าด้วยกันได้ และพร้อมสำหรับการผลิต สำหรับนักวิจัยมือใหม่ ผลกระทบที่กว้างขึ้นคือไม่จำเป็นต้องสร้างระบบ RAG ตั้งแต่เริ่มต้นอีกต่อไป แต่สามารถใช้ประโยชน์จากเฟรมเวิร์กเหล่านี้เพื่อสร้างต้นแบบ ทดลอง และมุ่งเน้นการวิจัยไปที่ส่วนประกอบหรือเทคนิคการเพิ่มประสิทธิภาพเฉพาะ ซึ่งจะช่วยเร่งการเรียนรู้และการมีส่วนร่วมในสาขานี้

การเลือกเครื่องมือที่เหมาะสม

การเลือกเครื่องมือที่เหมาะสมขึ้นอยู่กับความซับซ้อนของแอปพลิเคชัน โครงสร้างข้อมูล และทรัพยากรการคำนวณที่มีอยู่.[33] ควรพิจารณาปัจจัยต่างๆ เช่น ความง่ายในการใช้งาน, ความสามารถในการรวมเข้าด้วยกัน, การสนับสนุนจากชุมชน และคุณสมบัติเฉพาะ (เช่น การทำความเข้าใจเอกสารเชิงลึกสำหรับ PDF ที่ซับซ้อน, การจัดเรียงหลายเอเจนต์สำหรับเวิร์กโฟลว์ที่ซับซ้อน).[15]

เมื่อพิจารณาถึงจำนวนเฟรมเวิร์กที่มีอยู่มากมาย แนวทางปฏิบัติที่ดีที่สุดสำหรับผู้เริ่มต้นคือการเลือกเฟรมเวิร์กยอดนิยมที่มีเอกสารประกอบที่ดีหนึ่งหรือสองเฟรมเวิร์ก (เช่น LangChain หรือ LlamaIndex) ที่ให้ความสมดุลที่ดีระหว่างคุณสมบัติและการสนับสนุนจากชุมชน.[15] สิ่งนี้ช่วยให้สามารถเริ่มต้นได้อย่างรวดเร็วและทำการทดลองได้จริง ซึ่งเป็นสิ่งสำคัญสำหรับ "ผู้เริ่มต้นใน RAG" ที่กำลังทำ "วิจัย" การเลือกควรขับเคลื่อนด้วยงาน

เฉพาะที่ต้องการนำ RAG ไปใช้

ตาราง 5: เฟรมเวิร์ก RAG แบบ Open-Source ยอดนิยมและคุณสมบัติหลัก

ชื่อเฟรมเวิร์ก	GitHub Stars (โดยประมาณ)	Primary Focus (จุดเน้นหลัก)	Key Features (คุณสมบัติหลัก)
LangChain	⭐105k	Component chaining for general RAG applications.	Versatile Document Loaders, Efficient Text Splitters, Unified LLM Interface.[15]
Dify	⭐90.5k	Visual development for non-technical users and enterprise applications.	Visual workflow editor, Extensive model support, LLMOps.[19]
RAGFlow	⭐48.5k	Document processing for complex document handling.	Deep document understanding, Visual web interface, GraphRAG support.[15]
LlamaIndex	⭐40.8k	Data indexing for custom knowledge sources.	Flexible data connectors, Customizable indexing,

			Advanced retrieval mechanisms.[19]
Milvus	⭐33.9k	Vector storage for large-scale vector search.	Advanced vector search, Hybrid search capabilities, Horizontal scalability.[19]
mem0	⭐27.3k	Persistent memory for AI assistants with context retention.	Multi-level memory architecture, Automatic memory processing, Smart retrieval system.[19]
DSPy	⭐23k	Prompt optimization for systems requiring self-improvement.	Modular architecture, Automatic prompt optimization, Flexible pipeline design.[19]
Haystack	⭐20.2k	Pipeline orchestration for production applications.	Flexible component system, Technology-agnostic approach,

			Advanced retrieval methods.[19]
LightRAG	⭐14.6k	Performance for speed-critical applications.	Performance optimization, Simple architecture, Comprehensive retrieval.[19]
LLMWare	⭐12.7k	Resource efficiency for edge/CPU deployment.	Efficient model deployment, Comprehensive document processing, Multiple vector database options.[19]
txtai	⭐10.7k	All-in-one solution for streamlined implementation.	Embeddings database, Pipeline components, Workflow orchestration.[19]
RAGAS	⭐8.7k	Evaluation for RAG system testing.	Objective metrics, Test data generation, Specialized RAG metrics.[17]
R2R (RAG to	⭐6.3k	Agent-based	Multimodal

Riches)		RAG for complex queries.	ingestion, Hybrid search capability, Agentic reasoning.[19]
Ragatouille	⭐3.4k	Advanced retrieval for high precision search.	Late-interaction retrieval, Fine-tuning capabilities, Disk-based indices.[19]
FlashRAG	⭐2.1k	Research for experimentation and benchmarking.	Extensive dataset support, Algorithm implementations, Modular architecture.[19]

กรณีศึกษาและการประยุกต์ใช้ RAG ในโลกจริง

ส่วนนี้จะนำเสนอตัวอย่างที่เป็นรูปธรรมว่า RAG ถูกนำไปใช้อย่างประสบความสำเร็จในอุตสาหกรรมต่างๆ อย่างไร เพื่อแสดงให้เห็นถึงประโยชน์ในทางปฏิบัติและผลกระทบของมัน

ตัวอย่างการใช้งานในอุตสาหกรรมต่างๆ

RAG ได้รับการประยุกต์ใช้อย่างกว้างขวางในหลายอุตสาหกรรม แสดงให้เห็นถึงความสามารถในการแก้ปัญหาทางธุรกิจที่ซับซ้อน:

- **Chatbot สนับสนุนลูกค้า:**
    - **DoorDash:** ใช้ chatbot ที่ใช้ RAG พร้อม LLM guardrails และ judges เพื่อปรับปรุงการสนับสนุนการจัดส่ง โดยสรุปบทสนทนาและค้นหาฐานความรู้สำหรับบทความที่เกี่ยวข้อง.[34]

    - **LinkedIn:** รวม RAG เข้ากับ knowledge graph สำหรับการสนับสนุนด้านเทคนิคแก่ลูกค้า ลดเวลาแก้ไขปัญหาเฉลี่ยต่อปัญหาลง 28.6%.[34]
    - **กรณีการใช้งานทั่วไป:** ขับเคลื่อน chatbot ที่ชาญฉลาดขึ้น โดยดึงคำตอบโดยตรงจากบทความในศูนย์ช่วยเหลือ คู่มือผู้ใช้ หรือตั๋วสนับสนุนที่ผ่านมา ซึ่งนำไปสู่การตอบสนองที่รวดเร็วและแม่นยำยิ่งขึ้น และลดปริมาณตั๋ว.[18]
- **การค้นหาฐานความรู้ภายในองค์กร (สำหรับองค์กร):**
    - **Bell:** ใช้ RAG เพื่อปรับปรุงกระบวนการจัดการความรู้สำหรับนโยบายภายใน ทำให้พนักงานสามารถเข้าถึงข้อมูลที่ทันสมัยได้.[34]
    - **กรณีการใช้งานทั่วไป:** เปลี่ยนเนื้อหาภายใน (เอกสาร, นโยบาย, บันทึกการประชุม) ให้เป็นเครื่องมือค้นหาอัจฉริยะ ทำให้พนักงานสามารถถามคำถามด้วยภาษาธรรมชาติและได้รับคำตอบโดยตรง ซึ่งช่วยปรับปรุงประสิทธิภาพภายใน.[18] ตัวอย่างเช่น บริษัทที่ปรึกษาแห่งหนึ่งพบว่าคำถามภายในที่ซ้ำกันลดลง 50%.[18]
- **ศาสตราจารย์ AI (Harvard Business School):** ChatLTV ซึ่งเป็น chatbot อาจารย์ AI ที่ใช้ RAG ซึ่งได้รับการฝึกฝนจากเอกสารประกอบการเรียน ช่วยเหลือนักเรียนเกี่ยวกับแนวคิดและเรื่องการบริหาร.[34]
- **การสรุปวิดีโอ (Vimeo):** Chatbot ที่ใช้ RAG ช่วยให้ผู้ใช้สามารถสนทนากับวิดีโอ, สรุปเนื้อหา และเชื่อมโยงไปยังช่วงเวลาสำคัญ.[34]
- **รายงานการฉ้อโกงเชิงวิเคราะห์ (Grab):** ทำให้งานวิเคราะห์ประจำและงานสอบสวนการฉ้อโกงเป็นไปโดยอัตโนมัติ ประหยัดเวลา 3-4 ชั่วโมงต่อรายงาน.[34]
- **การสนับสนุนลูกค้าผู้บริหาร (Thomson Reuters):** ปรับปรุงการบริการลูกค้าโดยช่วยให้ผู้บริหารฝ่ายสนับสนุนเข้าถึงข้อมูลที่เกี่ยวข้องจากฐานข้อมูลที่คัดสรรได้อย่างรวดเร็ว.[34]

- **RAG สำหรับตารางข้อมูล (Pinterest):** ช่วยให้ผู้ใช้ข้อมูลภายในเขียนคำถาม SQL โดยการระบุตารางที่เกี่ยวข้อง.[34]
- **การจำแนกอุตสาหกรรมที่ปรับปรุงแล้ว (Ramp):** ใช้ RAG เพื่อปรับปรุงการจำแนกประเภทลูกค้าและย้ายไปยังระบบมาตรฐาน (North American Industry Classification System - NAICS).[34]
- **การค้นหานโยบายภายใน (Royal Bank of Canada - RBC):** Arcane ซึ่งเป็นระบบ RAG ชี้นำผู้เชี่ยวชาญของธนาคารไปยังนโยบายที่เกี่ยวข้องมากที่สุดที่กระจัดกระจายอยู่ทั่วแพลตฟอร์มเว็บภายใน ช่วยเพิ่มประสิทธิภาพการทำงาน.[34]

การใช้งานจริงที่หลากหลาย [18] แสดงให้เห็นถึงความหลากหลายและคุณค่าในทางปฏิบัติของ RAG ในอุตสาหกรรมต่างๆ ตั้งแต่การสนับสนุนด้านเทคนิคไปจนถึงการเงินและการศึกษา สิ่งนี้บ่งชี้ว่า RAG ไม่ใช่แค่ความก้าวหน้าทางทฤษฎีเท่านั้น แต่เป็นเทคโนโลยีที่เติบโตเต็มที่และสามารถนำไปใช้งานได้จริงในการแก้ปัญหาทางธุรกิจที่จับต้องได้ โดยเฉพาะอย่างยิ่งที่เกี่ยวข้องกับชุดข้อมูลขนาดใหญ่ ข้อมูลที่เป็นกรรมสิทธิ์ หรือข้อมูลที่เปลี่ยนแปลงอย่างรวดเร็ว แนวโน้มที่ชัดเจนคือองค์กรต่างๆ กำลังใช้ประโยชน์จาก RAG เพื่อทำให้งานที่ต้องใช้ความรู้จำนวนมากเป็นไปโดยอัตโนมัติ ปรับปรุงการตัดสินใจ และเพิ่มประสบการณ์ผู้ใช้/พนักงาน โดยมุ่งไปสู่การดำเนินงานที่ชาญฉลาดและมีประสิทธิภาพมากขึ้น
ตัวอย่างในโลกแห่งความเป็นจริงหลายกรณี (DoorDash, LinkedIn, Grab, Harvard Business School) กล่าวถึงการใช้ "LLM Guardrails" หรือ "LLM Judge" สำหรับการตรวจสอบและประเมินคุณภาพของระบบ RAG อย่างต่อเนื่องในการผลิต.[34] สิ่งนี้เผยให้เห็นรูปแบบที่ซ่อนอยู่: การนำ RAG ไปใช้งานในโลกแห่งความเป็นจริงที่ประสบความสำเร็จนั้นนอกเหนือไปจากการพัฒนาเริ่มต้น และต้องการกลยุทธ์ MLOps/LLMOps ที่แข็งแกร่ง การประเมินคุณภาพอย่างต่อเนื่องเป็นสิ่งสำคัญสำหรับการรักษาประสิทธิภาพ ป้องกันการถดถอย และรับรองการปฏิบัติตามข้อกำหนดในสภาพแวดล้อมจริง สิ่งนี้บ่งชี้ว่าสำหรับนักวิจัยที่มุ่งเน้นการใช้งานจริง การทำความเข้าใจวงจรชีวิตของระบบ RAG รวมถึงการ

ตรวจสอบหลังการปรับใช้ มีความสำคัญเท่ากับการสร้างระบบเริ่มต้น

สรุปและทิศทางงานวิจัยในอนาคต

RAG ได้พิสูจน์แล้วว่าเป็นเทคนิคที่ทรงพลังในการปรับปรุงความสามารถของ LLMs โดยการเชื่อมโยงกับแหล่งความรู้ภายนอก ช่วยลดการหลอนข้อมูลและให้ข้อมูลที่ทันสมัยและตรวจสอบได้ การสร้างระบบ RAG ที่มีประสิทธิภาพต้องอาศัยความเข้าใจอย่างลึกซึ้งในสถาปัตยกรรม การเตรียมข้อมูล การดึงข้อมูล การเสริมข้อมูล การสร้างข้อความ และที่สำคัญที่สุดคือการประเมินอย่างต่อเนื่อง

ข้อเสนอแนะสำหรับนักวิจัยมือใหม่

สำหรับนักวิจัยมือใหม่ที่ต้องการเจาะลึก RAG ข้อเสนอแนะต่อไปนี้สามารถเป็นแนวทางในการเดินทาง:

- **เริ่มต้นอย่างง่าย:** เริ่มต้นด้วยการใช้งาน RAG พื้นฐานโดยใช้เฟรมเวิร์กยอดนิยมเช่น LangChain หรือ LlamaIndex เพื่อทำความเข้าใจขั้นตอนการทำงานหลัก.[11]
- **มุ่งเน้นคุณภาพข้อมูล:** การเตรียมข้อมูลอย่างพิถีพิถัน (การโหลด, การแบ่งส่วน, การสร้าง embedding) เป็นรากฐานสำคัญ หลักการ "ข้อมูลขยะเข้า ข้อมูลขยะออก" มีผลอย่างมากกับ RAG.[8]
- **ทำซ้ำและประเมินผล:** ใช้รอบการพัฒนาแบบวนซ้ำ การประเมินอย่างเข้มงวดโดยใช้เมตริกทั้งเชิงปริมาณและเชิงคุณภาพเป็นสิ่งสำคัญในการระบุคอขวดและเพิ่มประสิทธิภาพ.[13] เปลี่ยนตัวแปรทีละตัวระหว่างการทดสอบ.[17]
- **เชี่ยวชาญ Prompt Engineering:** ทำความเข้าใจว่าวิธีการนำเสนอบริบทที่ดึงมาให้ LLM มีผลอย่างมากต่อผลลัพธ์สุดท้าย ทดลองกับการออกแบบ prompt ที่แตกต่างกันและเทคนิคขั้นสูงเช่น CoT.[21]
- **สำรวจเทคนิคขั้นสูงทีละน้อย:** เมื่อคุ้นเคยกับ RAG พื้นฐานแล้ว ให้สำรวจกลยุทธ์ขั้นสูงเช่น reranking, query expansion หรือสถาปัตยกรรม RAG ที่แตกต่างกัน (Hybrid, Agentic, Graph) โดย

อิงตามความซับซ้อนของงานเฉพาะของคุณ.[1]

- **มีส่วนร่วมกับชุมชน:** ติดตามความคืบหน้าของ RAG ที่เปลี่ยนแปลงอย่างรวดเร็วโดยติดตามชุมชนวิจัย, จดหมายข่าว และบล็อก.[29]
- **กำหนด "งานเฉพาะ" ของคุณให้ชัดเจน:** "วิธีการที่มีประสิทธิภาพดีที่สุด" ขึ้นอยู่กับงานอย่างมาก กำหนดปัญหาการวิจัย ลักษณะข้อมูล และผลลัพธ์ที่ต้องการให้ชัดเจนก่อนที่จะลงลึกในการนำไปใช้งาน.

ทิศทางงานวิจัยในอนาคต

RAG เป็นสาขาที่กำลังพัฒนาอย่างรวดเร็ว โดยมีแนวโน้มการวิจัยที่น่าสนใจหลายประการ:

- **Agentic RAG และ Multi-Agent Systems:** การพัฒนาเพิ่มเติมในการตัดสินใจแบบไดนามิก, การวางแผน และการทำงานร่วมกันของหลายเอเจนต์สำหรับเวิร์กโฟลว์ที่ซับซ้อน.[30]
- **Multimodal RAG:** การขยาย RAG นอกเหนือจากข้อความเพื่อรวมและให้เหตุผลกับข้อมูลรูปภาพ, เสียง, วิดีโอ และ 3D.[4]
- **ประสิทธิภาพและการปรับขนาด:** การเพิ่มประสิทธิภาพ, เวลาแฝง และต้นทุนสำหรับการปรับใช้ในระดับการผลิตขนาดใหญ่ รวมถึงเทคนิคเช่น KV-cache reuse.[9]
- **ความแข็งแกร่งและการตีความ:** การแก้ไขจุดล้มเหลว, การปรับปรุงการตรวจจับข้อมูลนอกโดเมน และการเพิ่มความโปร่งใสและการอธิบายของระบบ RAG.[9]
- **การเพิ่มประสิทธิภาพอัตโนมัติ:** การพัฒนากรอบการทำงานที่สามารถเพิ่มประสิทธิภาพส่วนประกอบ RAG โดยอัตโนมัติ (เช่น การแบ่งส่วน, โมเดล embedding, กลยุทธ์ prompt) โดยอิงตามเมตริกประสิทธิภาพ.[19]
- **ข้อพิจารณาด้านจริยธรรม:** การรับรองความเป็นธรรม, การลดอคติ และการรักษาความเป็นส่วนตัวและความปลอดภัยของข้อมูลในการปรับใช้ RAG.[5]

ทิศทางการวิจัยในอนาคตเหล่านี้เน้นย้ำถึงแนวโน้มที่ชัดเจนไปสู่ระบบ

RAG ที่ชาญฉลาด หลากหลาย และแข็งแกร่งยิ่งขึ้น จุดเน้นกำลังเปลี่ยนจากการทำงานพื้นฐานไปสู่การจัดการกับความซับซ้อนในโลกแห่งความเป็นจริง เช่น ข้อมูลหลายโมดอล ความสามารถในการปรับขนาด และการนำไปใช้งานอย่างมีจริยธรรม ผลกระทบที่กว้างขึ้นคือ RAG ไม่ใช่เพียงเทคนิคชั่วคราว แต่เป็นกระบวนทัศน์พื้นฐานสำหรับการสร้างแอปพลิเคชัน AI ที่มีความสามารถสูงและน่าเชื่อถือ ซึ่งสามารถโต้ตอบและทำความเข้าใจโลกในรูปแบบที่ซับซ้อนมากขึ้น สำหรับนักวิจัยมือใหม่ สิ่งนี้มอบโอกาสที่หลากหลายสำหรับปัญหาที่เปิดกว้างและการมีส่วนร่วมที่มีผลกระทบ

ผลงานที่อ้างอิง

1. Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/html/2504.14891v1
2. A Survey on Knowledge-Oriented Retrieval-Augmented Generation - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/html/2503.10677v2
3. A Simple Guide To Retrieval Augmented Generation Language ..., เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.smashingmagazine.com/2024/01/guide-retrieval-augmented-generation-language-models/
4. What is Retrieval-Augmented Generation (RAG)? | Google Cloud, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://cloud.google.com/use-cases/retrieval-augmented-generation
5. What is RAG (Retrieval Augmented Generation)? | IBM, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.ibm.com/think/topics/retrieval-augmented-generation
6. Understanding RAG: Retrieval-Augmented Generation

Explained - 6clicks, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.6clicks.com/resources/blog/understanding-rag-retrieval-augmented-generation-explained
7. Introduction to Retrieval Augmented Generation (RAG) - Redis, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://redis.io/glossary/retrieval-augmented-generation/
8. RAG 101: What is RAG and why does it matter? - Codingscape, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://codingscape.com/blog/rag-101-what-is-rag-and-why-does-it-matter
9. How to Get Enterprise RAG Right | 5 Key Principles - Pryon, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.pryon.com/guides/how-to-get-enterprise-rag-right
10. QuIM-RAG: Advancing Retrieval-Augmented Generation with Inverted Question Matching for Enhanced QA Performance - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/html/2501.02702v1
11. Build a Retrieval Augmented Generation (RAG) App: Part 1 ..., เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://js.langchain.com/docs/tutorials/rag/
12. Building A RAG System with Gemma, MongoDB and Open Source ..., เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://huggingface.co/learn/cookbook/rag_with_hugging_face_gemma_mongodb
13. Retrieval-augmented generation (RAG) fundamentals - Databricks Documentation, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://docs.databricks.com/aws/en/generative-ai/tutorials/ai-cookbook/fundamentals-retrieval-augmented-generation
14. Building a RAG System With Google's Gemma, Hugging Face and MongoDB, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.mongodb.com/developer/products/atlas/gemma-mongodb-huggingface-rag/
15. 25+ Best Open Source RAG Frameworks in 2025 - Signity

Solutions, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.signitysolutions.com/blog/best-open-source-rag-frameworks
16. arxiv.org, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/abs/2501.07391
17. RAG systems: Best practices to master evaluation for accurate and ..., เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://cloud.google.com/blog/products/ai-machine-learning/optimizing-rag-retrieval
18. 10 Most Useful RAG Application & Use Cases [Real World Example], เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://botpenguin.com/blogs/most-useful-rag-application-and-use-cases
19. 15 Best Open-Source RAG Frameworks in 2025 - Firecrawl, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.firecrawl.dev/blog/best-open-source-rag-frameworks
20. arxiv.org, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/abs/2501.14342
21. Emulating Retrieval Augmented Generation via Prompt Engineering for Enhanced Long Context Comprehension in LLMs - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/html/2502.12462v1
22. Enhancing Retrieval-Augmented Generation: A Study of Best Practices - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/html/2501.07391v1
23. HyperRAG: Enhancing Quality-Efficiency Tradeoffs in Retrieval-Augmented Generation with Reranker KV-Cache Reuse - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/html/2504.02921v1
24. Cache-Craft: Chunk-Level KV Cache Reuse for Faster and Efficient RAG (SIGMOD 2025), เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.reddit.com/r/LocalLLaMA/comments/1j2pm6n/cac

hecraft_chunklevel_kv_cache_reuse_for_faster/
25. arxiv.org, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/abs/2402.01733
26. Enhancing RAG Systems with Advanced Prompting Techniques - Oracle Blogs, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://blogs.oracle.com/ai-and-datascience/post/enhancing-rag-with-advanced-prompting
27. Prompt engineering for RAG - OpenAI Developer Community, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://community.openai.com/t/prompt-engineering-for-rag/621495
28. The Prompt Report: A Systematic Survey of Prompt Engineering Techniques - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/html/2406.06608v6
29. Need some clarifications on RAG (Retrieval-Augmented Generation) - Unified Endpoint Management Community | Hexnode Connect, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.hexnode.com/forums/topic/need-some-clarifications-on-rag-retrieval-augmented-generation/
30. [2501.09136] Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/abs/2501.09136
31. Agentic RAG explained: Smarter retrieval with AI agents - Glean, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.glean.com/blog/agentic-rag-explained
32. Overview："Agentic Retrieval-Augmented Generation: A Comprehensive Survey", เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://dev.to/foxgem/overviewagentic-retrieval-augmented-generation-a-comprehensive-survey-34i6
33. Advanced RAG Methods: Simple, Hybrid, Agentic, Graph Explained, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://blog.premai.io/advanced-rag-methods-simple-hybrid-agentic-graph-explained/

34. 10 RAG examples and use cases from real companies - Evidently AI, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.evidentlyai.com/blog/rag-examples
35. RAGXplain: From Explainable Evaluation to Actionable Guidance of RAG Pipelines - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/html/2505.13538v1
36. RAGXplain: From Explainable Evaluation to Actionable Guidance of RAG Pipelines - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/pdf/2505.13538
37. arxiv.org, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/pdf/2401.05856
38. Build a Retrieval Augmented Generation (RAG) App: Part 2 | 🦜 LangChain, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://python.langchain.com/docs/tutorials/qa_chat_history/
39. Seven Failure Points When Engineering a Retrieval Augmented Generation System - arXiv, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/html/2401.05856v1
40. What resources do you use for staying up-to-date with research ..., เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://www.reddit.com/r/Rag/comments/1gmf8hi/what_resources_do_you_use_for_staying_uptodate/
41. arxiv.org, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/abs/2503.18016
42. arxiv.org, เข้าถึงเมื่อ พฤษภาคม 27, 2025 https://arxiv.org/abs/2503.14649